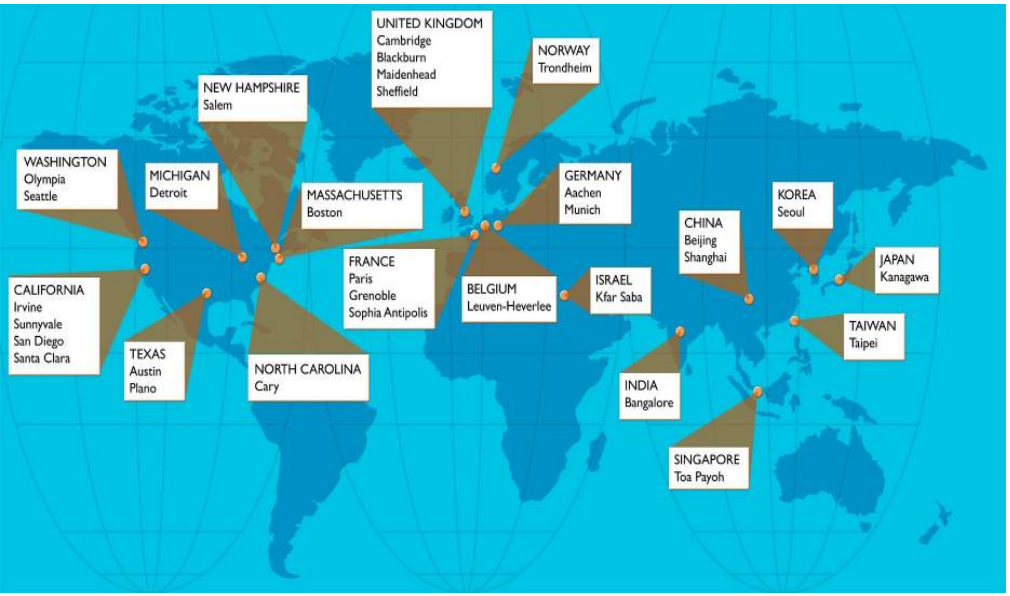
Agenda

- **Introduction to ARM Ltd**
- The ARM Architecture
  - Introduction
  - Programmer's Model
  - Instruction Sets
- ARM Processor Cores
  - ARM7TDMI family
  - ARM9TDMI family
  - ARM10E family
  - ARM11 family
  - Cortex family
  - Other processors
- ARM System Design
  - AMBA
  - Debug and Trace
- Writing Software for ARM Processors
  - Intro to RealView Development Suite
  - Exception Handling
  - Embedded Software Development
  - Development Platforms

About ARM



- **Founded in Nov 1990, UK**
- **Well known for RISC (Reduced Instruction Set Computer) based architecture**
- ***Top 10 most Significant Electronics Companies in last 30 yrs***
- **ARM architecture is the most successful and dominant 32-bit architecture**
- **Leading IPs provider**

ARM's Activities

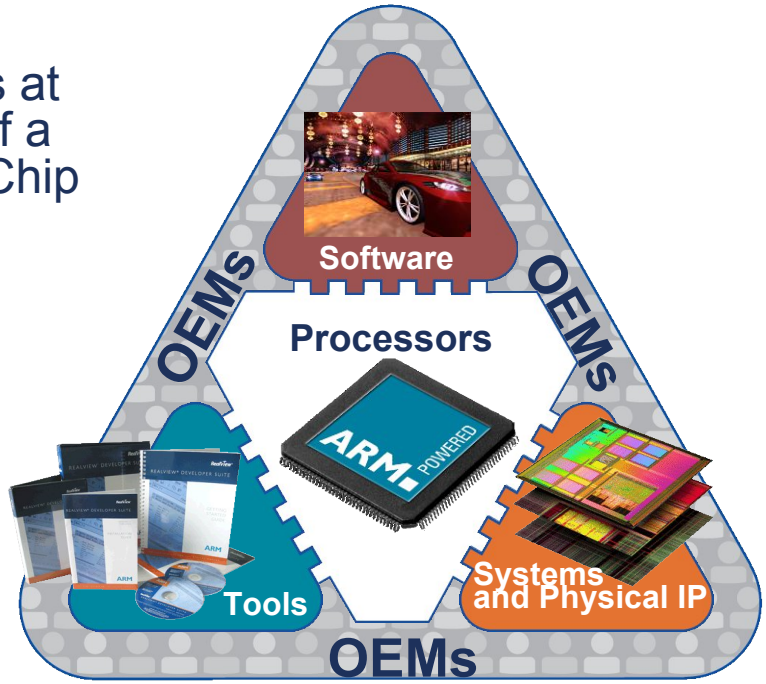
Processors
System Level IP:
Data Engines
Fabric

Physical IP

The ARM Strategy

The SoC must enable the device manufacturer to produce highly desirable products

The CPU is at the heart of a System on Chip



Software to bring the system alive and to maximise the performance of the system

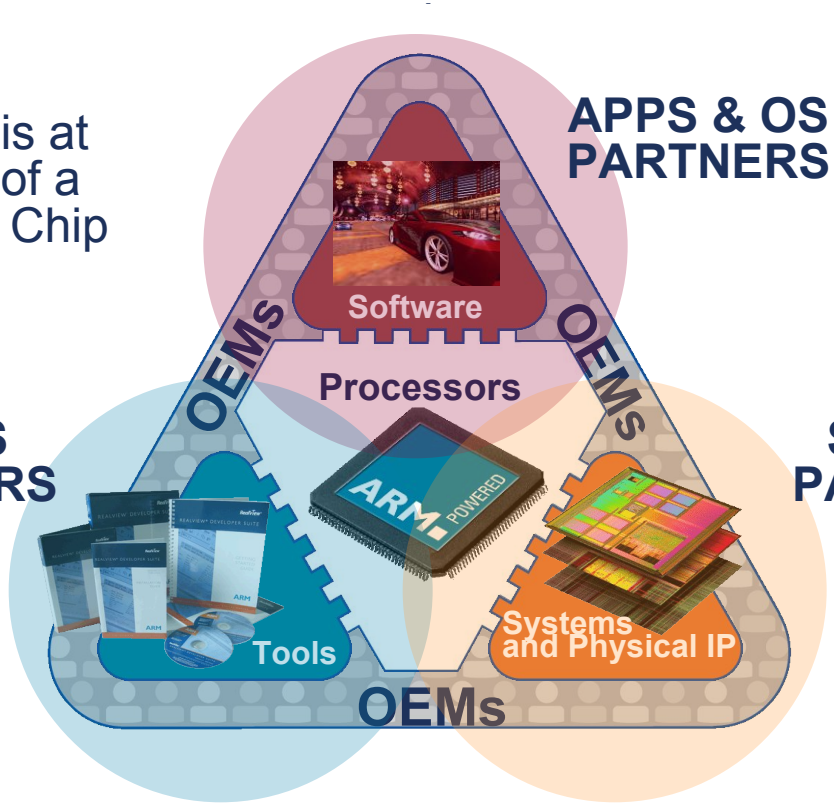
Software tools, ESL tools and models to target software onto the processors in the system

System on Chips require high performance Fabric along with high quality Physical IP

The ARM Strategy

Our strategy is to create a partnership with our customers and broader community of third parties, to enable the creation of end products more efficiently through ARM than from any other source

The CPU is at the heart of a System on Chip

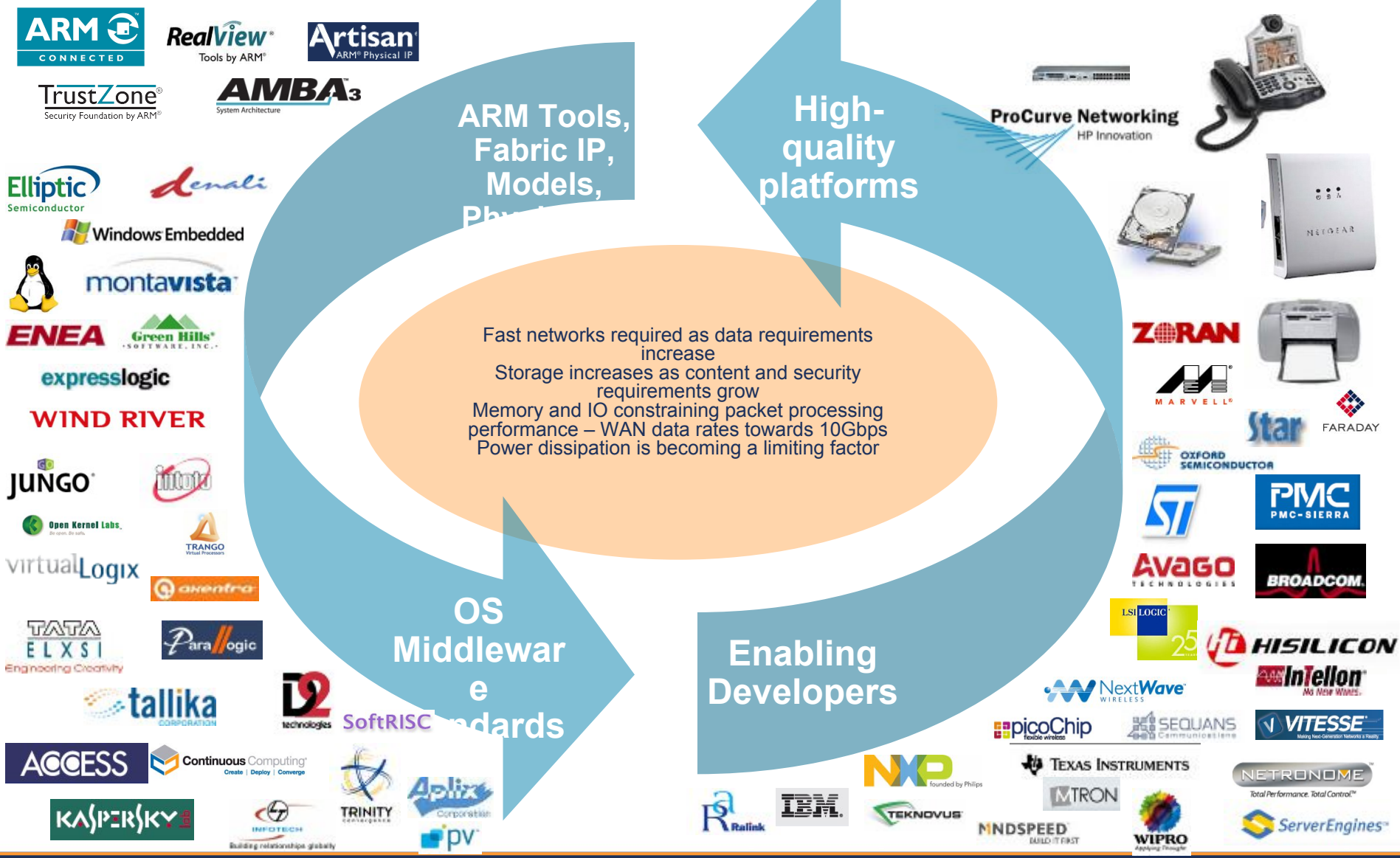


We work with Operating Systems suppliers to ensure support for the processor architecture and enable a wide community of software suppliers

We enable a wide community of software tools providers to give the silicon Partners and OEM's wide choice

We partner with EDA and Silicon companies to provide diversity of SoC implementation and manufacturing choice

ARM customers and partners (1)



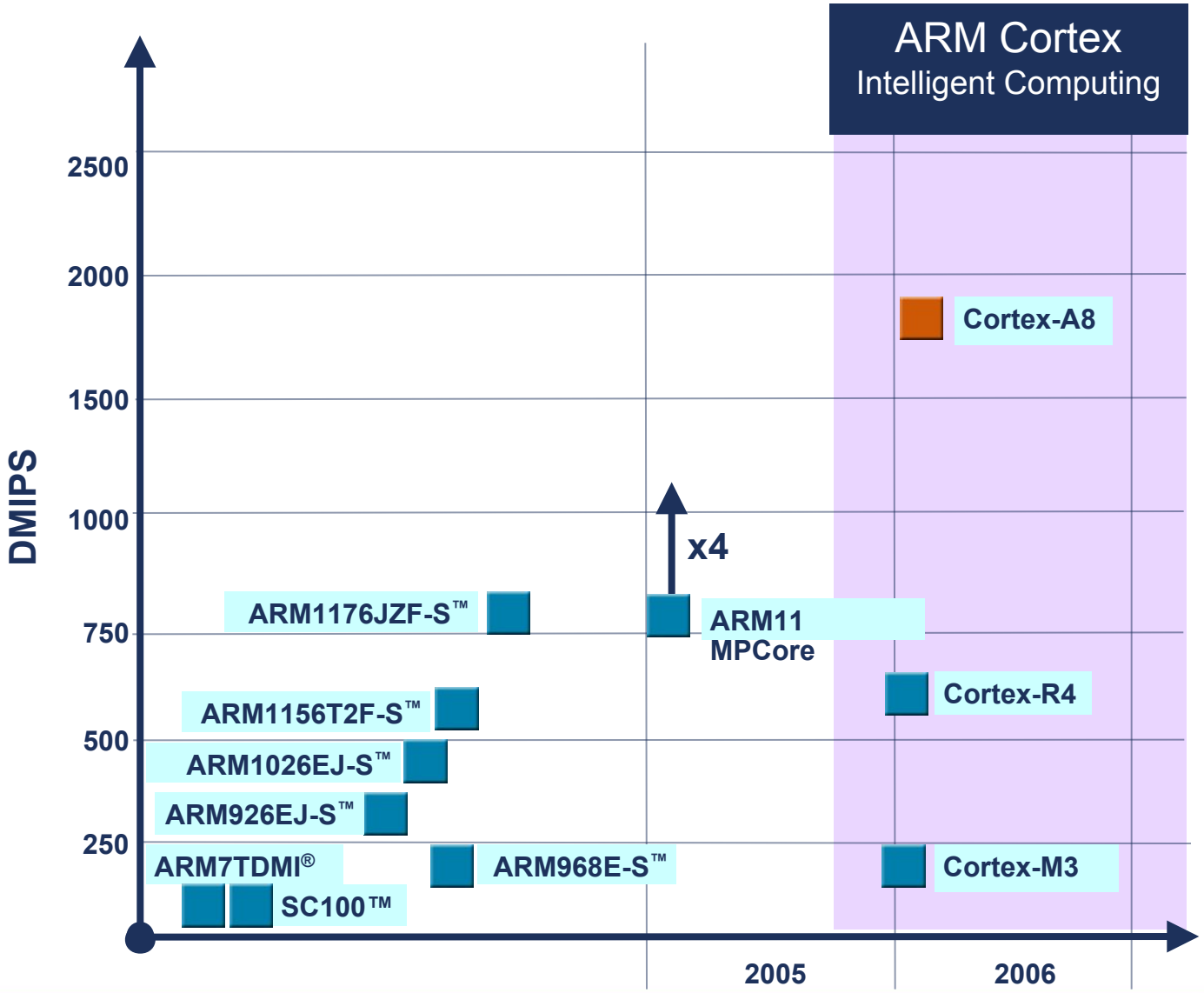
ARM customers and partners (…)

...+ ... + ... => ARM Community

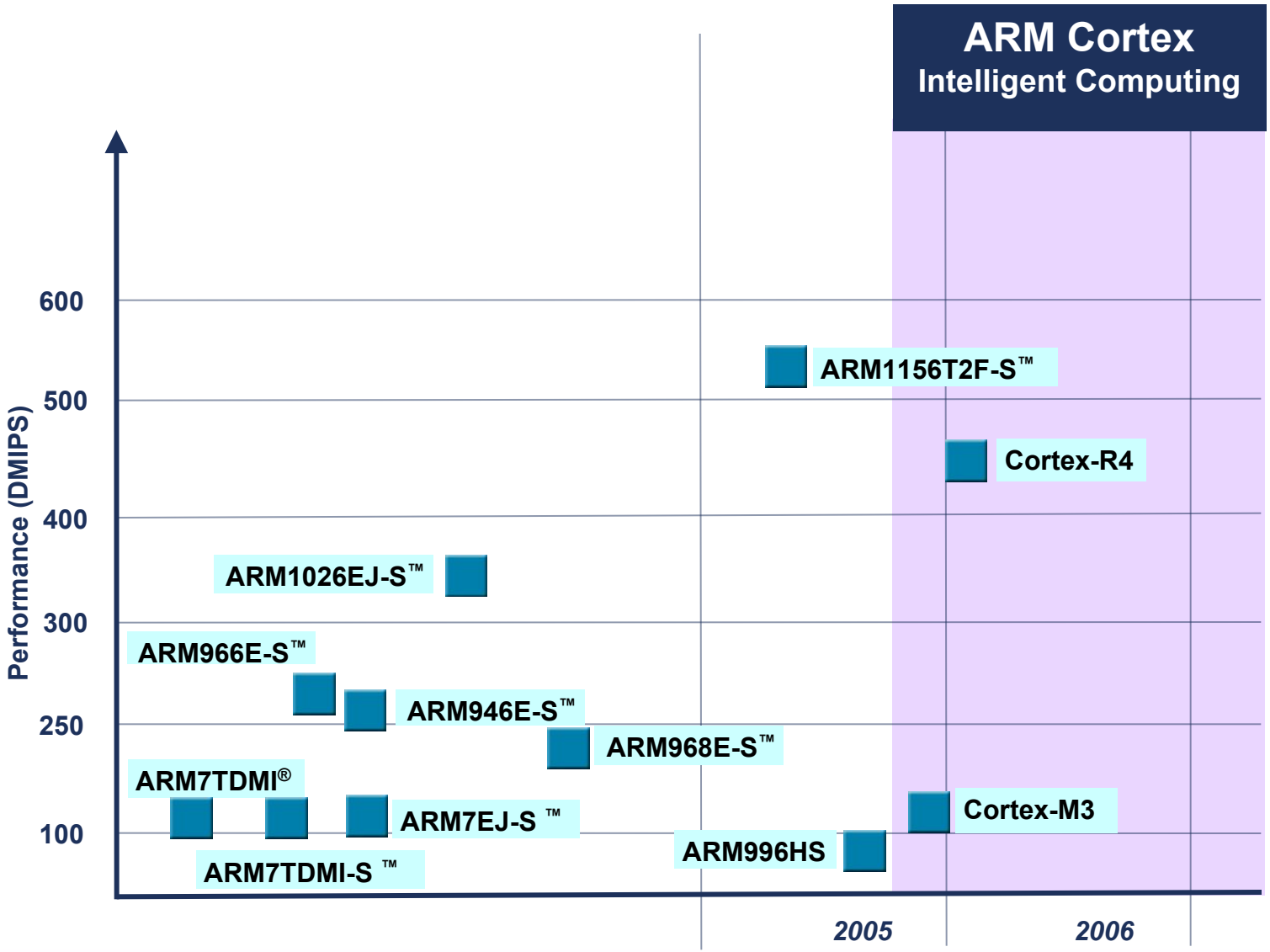
Our Vision

ARM designs technology that lies at the heart of advanced digital products

Applications Processor Roadmap



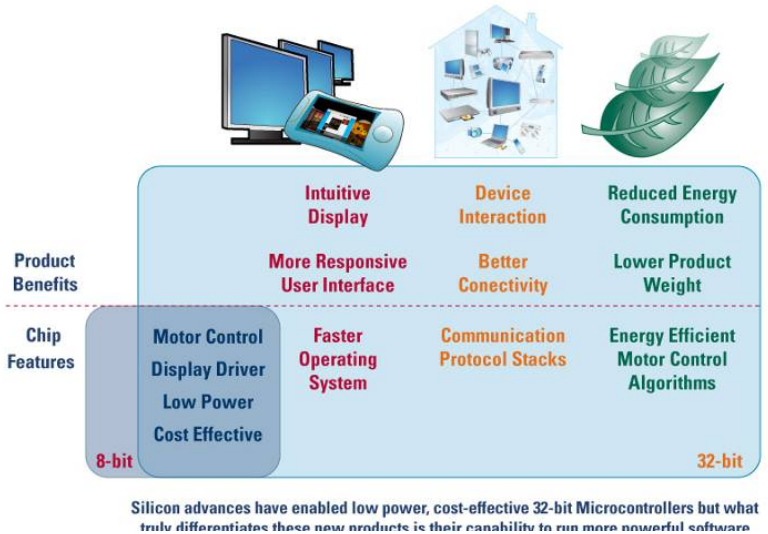
Embedded Processor Roadmap

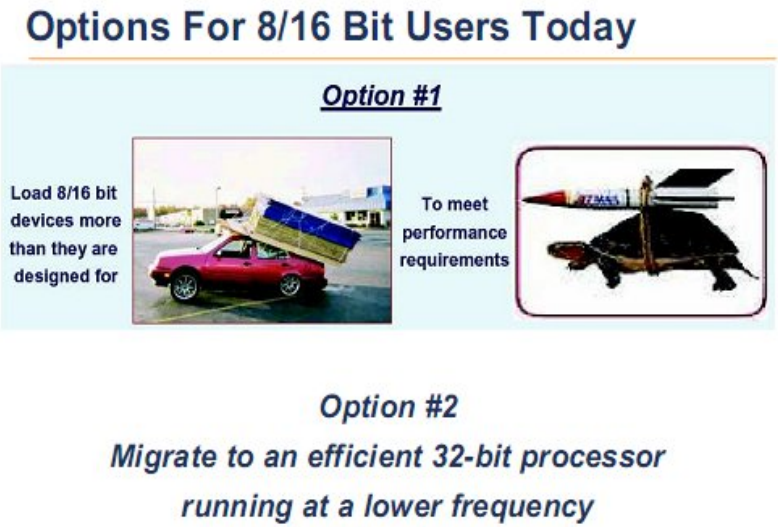


Why ARM (1)

- **As an example, 8051 reaches limits**
  - Limited support for modern techniques
    - Little code reuse, difficult to move code between 8051 devices
    - Lack of good RTOS support
  - C programming new to 8051
    - Assembly language skills not common today
  - Legacy application code
    - Difficult to maintain
    - Very hard to port new 8051 devices
    - Easier to move to new modern architecture like the Cortex-M3
  - 8051 offers few debugging options
    - Monitor programs like Keil MON51 are required
    - JTAG debugging is available on few devices

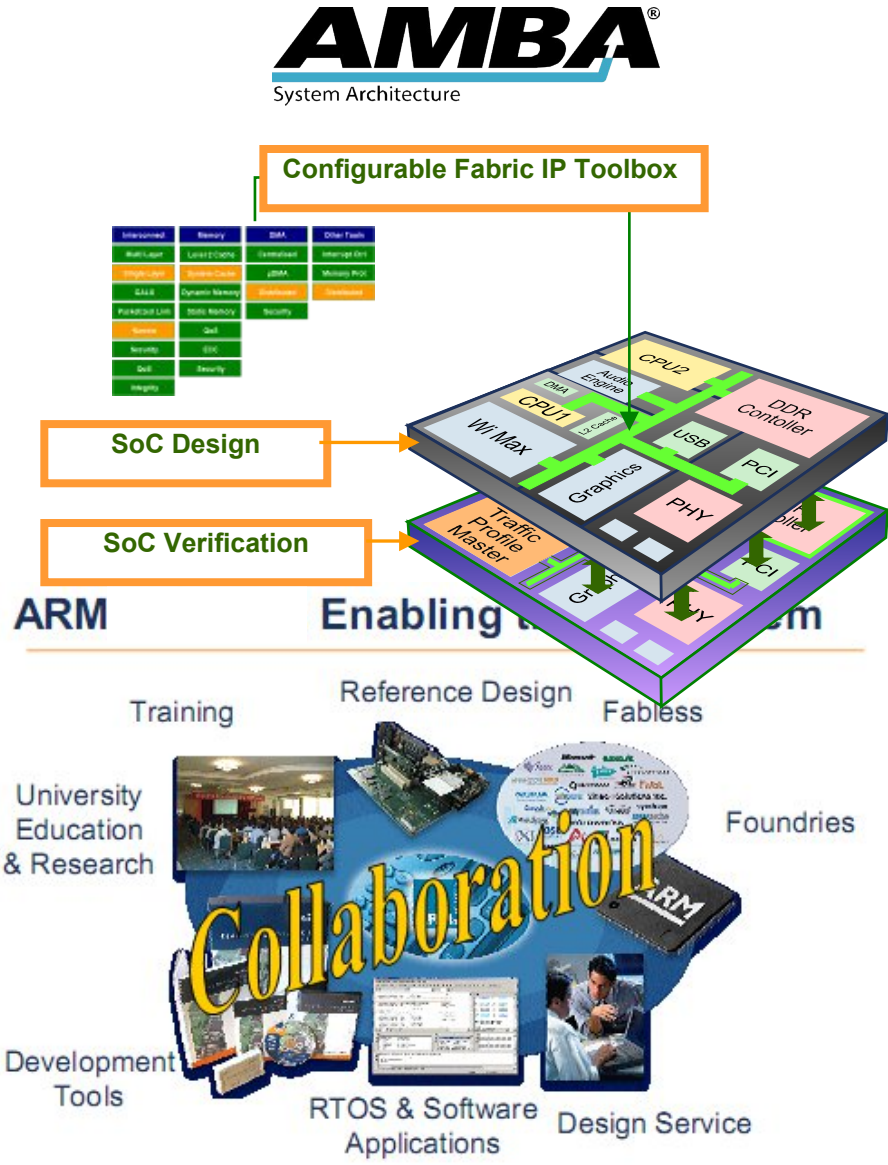
- **We need 32bit processors (ARM)**





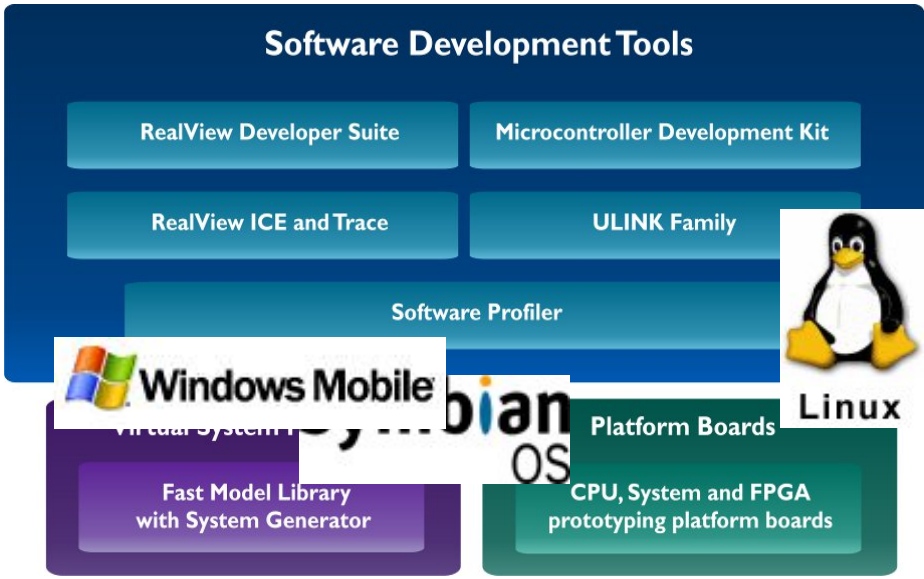
Why ARM (2)

- **Complete solution for SoC design**
  - Advanced 32-bit RISC processors, cutting edge AMBA, intelligent memory management, rich IPs and best development tools => highest system integration capability (hight speed and small die size).
- **High performance at reasonable price by the means of various excellent solutions: Thumb-2, IEM, … => lowest power consumption, very important feature of advanced digital devices**
- **A widest range of Prime Cells of high performance together with leading industry tools for ARM 32-bit architecture**



Why ARM (3)

- **ARM architecture based processors receive widest and best support among processors**
  - By ARM, ARM partners and ARM community (Microsfoft, Linux, Apple ….).
  - Provide widest options for supporting the development of OS based applications, e.g, Windows Mobile, Windows CE, ….
- **Significantly reduce Time-To-Market**
  - Strong and wide support of trusted and world-class tools provided by ARM and ARM community
  - Provide reusable IPs, platforms, … and leading software development tools such as MDK, RVDS, ...



Why ARM (4)

- **ARM commits to non-stop R&D**
  - Aim to better provide and meet market requirements and capture future trends.
  - Proved by ARM success in the development of advanced technologies such as IPs, Neon, Multi media, TrustZone, VFP, Thumb-2, Java Jazelle …
- **In addition to design and technical supports, ARM provides trusted training programs across the world**
  - Countries possessing developed digital technology and potential for advanced digital technology, e.g., UK, Germany, South Korea, Taiwan, Vietnam …
  - In Vietnam, ARM authorised PTIT (Posts and Telecommunications Institute of Technology) to run three ARM courses

OptimoDE™ ARM® Optimal Data Engine technology

Cortex

TrustZone

RealView® Tools by ARM®

Neon

Jazelle® ARM® Acceleration Technology for the Java™ Platform

Swerve

MBX

PrimeXsys™ ARM® Extendible System Technology

IEM

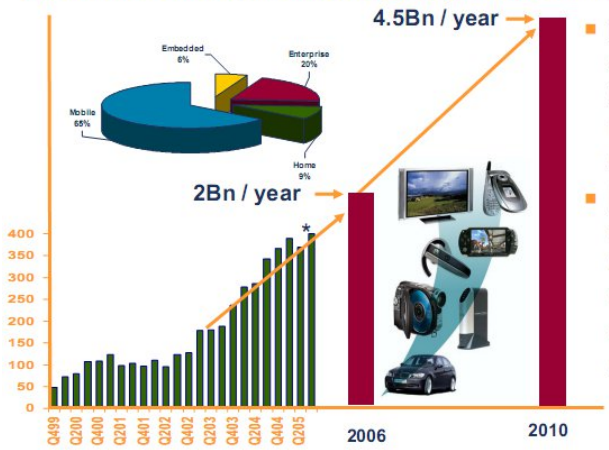
Applications – Products - Technology

ARM technology: Facts and Figures

- **ARM technology is used in > 90% of world wide mobiles**
  - At least 90% mobile phones use at least one ARM processor
- **9 mil chips shipped every day**
  - **10x PC shipment**
  - Expect over 5 bil cores shipped in 2011
  - Fast annual growth in non-mobile
    - MCU: highest volume contributor after wireless handsets
- **Leading provider of 32-bit embedded RISC microprocessors**
  - **75% of market**
- **ARM MCUs increase 2.4x per year**
  - **Vision 2018: ARM processors dominate the MCU market (60% - 70%)**
  - De-facto standard architecture for embedded applications

Driving Momentum: 4.5Bn Units by 2010

Over 2 Billion ARM based devices were shipped in 2006 ..... 6,000 every second

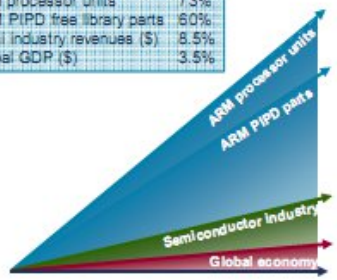


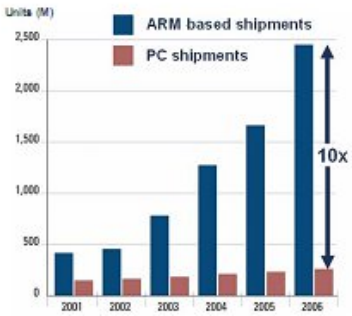
- Distributors sees the strong market momentum of ARM
- Distributors are riding the wave of consumer demand for ARM IP

Growing Faster than the Market

- Together we are growing faster than the rest of the market
- Semiconductor industry outpaces the global economy
- ARM processor based semiconductors growing at 9x growth of industry
- ARM Physical IP based semiconductors growing at 7x growth of industry

Compound annual growth rates	
ARM processor units	73%
ARM PIPD free library parts	60%
Semi industry revenues ($)	8.5%
Global GDP ($)	3.5%





Cool Products

Blackberry Bold
Marvell "Tavor" PXA930
ARM Architecture Based – Xscale processor

Innovations for learning - Teachermate
ARM9 Processor

D-Link DNS323 Network Storage Enclosure
Marvell 88F5181 Soc – ARM9 processor

iRiver Unit 2 Multimedia Home Networking device
Telechips and Samsung – ARM9E + ARM11 Processors

Samsung SMT-H30560 Cable STB
Conexant CX2417X – ARM920T Processor

Sonosite MTurbo (Portable Ultrasound Device)
Texas Instruments TMS320DM644x - ARM926 Processor

Embedded Automation – mPanel (Digital Home Device)
ARM architecture-based – Marvel XScale

Sunlink International - SunView PMP + Projector
Samsung S3C244A – ARM9 Processor

Garmin Nuvi 205
ST Cartesio Processor - ARM926

Everex Cloudbook UMPC
GCT Semiconductor – ARM9 Processor

Importek Apollo VoIP Video Phone
ARM9 + Marvell Xscale processor

Thomson WiFi Tablet
TI DaVinci TMS320DM6441 – ARM926EJ-S

Artega - Artega GT (Dual-Dashboard Display)
Fujitsu MB86R01 "Jade" graphics controller
ARM926EJ-s + Jazelle Java Acceleration Technology

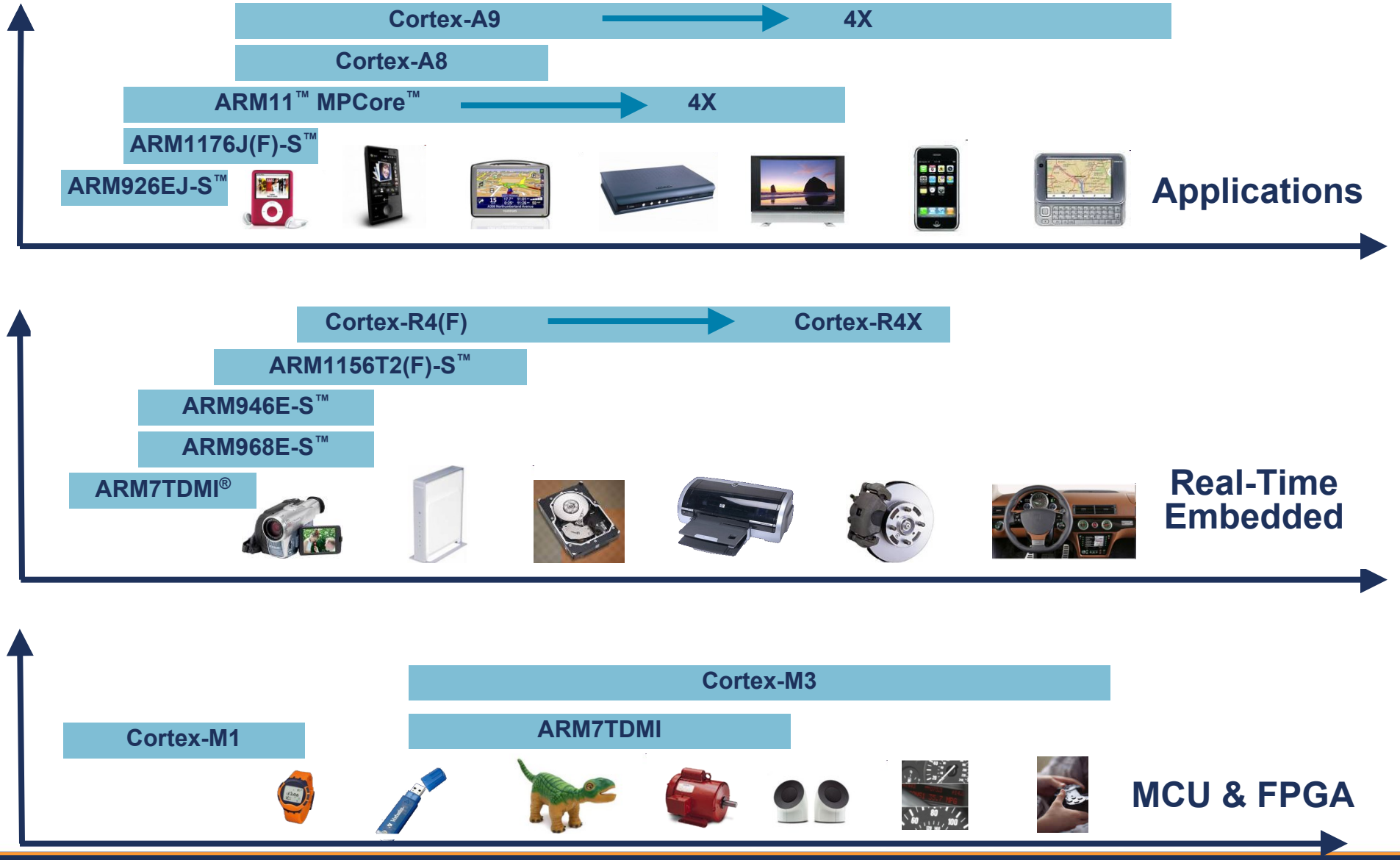
VivoPay – Vivo Kiosk
ARM Powered

iRiver NV Life PMP
Magic Eyes - ARM926EJ +
ARM946E

Custom Engineering - TK300II Desktop Ticket Printer
ARM Processor (266MHz)

Profiles – Segmenting the Architecture

Processors Across Applications



ARM and End-Product Trends?

GPS

Biometrics

20 GB HD

3.5G (HSDPA or 1xEV-DO rev A)

Stereo Headset

Bluetooth / UWB

- 1 billion transistor chips
- 'Tiger' 1GHz or MPcore 65nm
- VGA H.264 encode
- GPU (PSP level performance)
- 256MB DRAM, 64MB Flash
- WVGA screen, (852 x 480)
- 4 Mega Pixel Camera

NFC / RFID

TV out

PC / Mac

512 MB Memory card

DMB (Digital Mobile Broadcast)

MediaFLO

ISDB-T

ARM Technology is Designed To Help

OptimoDE™
ARM® Optimal Data Engine technology

TrustZone

Cortex

jazelle®
ARM® Acceleration Technology
for the Java™ Platform

Neon

MBX

PrimeXsys™
ARM® Extendible System Technology

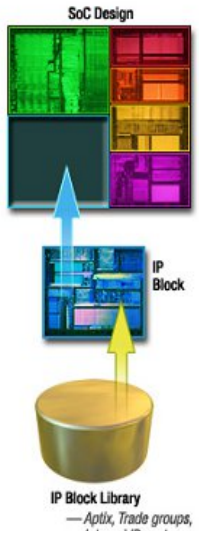
Swerve

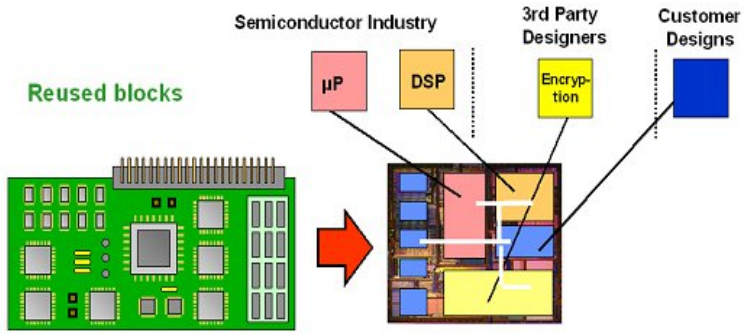
IEM

RealView®
Tools by ARM®

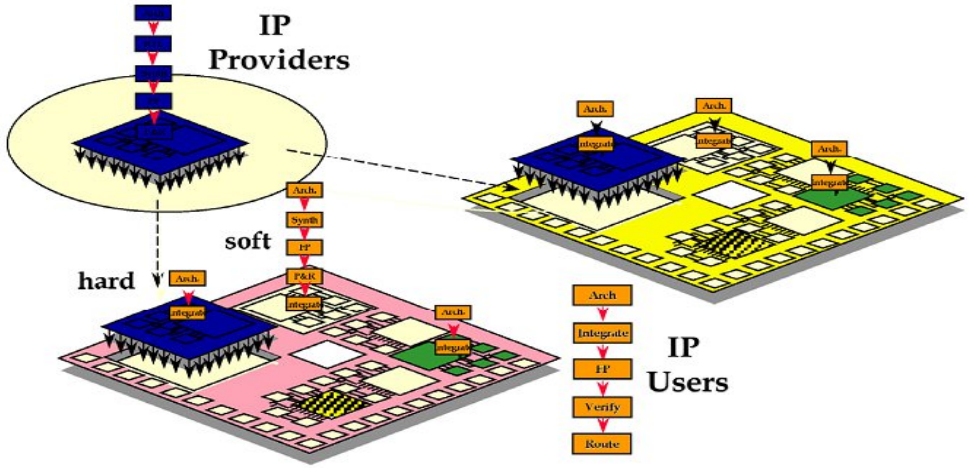
ARM as SoC

- **ARM processors integrate everything that you need to build a System-on-Chip:**
  - Fast processor cores
  - High speed and performance bus: AMBA
  - A variety of peripherals
    - Traditional: GPIO, timers, ADCs, DACs PWMs, etc.
    - Advanced: CANs, Internet, LCD/touch screen controllers, DSP, (N)VICs, VFP SAT, etc.
  - PrimeCells for your further development
- **Advanced memory systems (caches, MPU, MMU) to support OSs (e.g., Linux, Symbian, Windows CE, Windows Mobile) and security (TrustZone)**
- **Support multi-cores (up to 15 co-processors)**
- **Support FPGA (FPGA targeted)**
  - The ARM Cortex™-M1 processor is the first ARM processor designed specifically for implementation in FPGAs. The Cortex-M1 processor targets all major FPGA devices and includes support for leading FPGA synthesis tools, allowing the designer to choose the optimal implementation for each project.





Definition: integration of a complete system onto a single IC



SOC is industry trend

ASIC/ASSP

Today

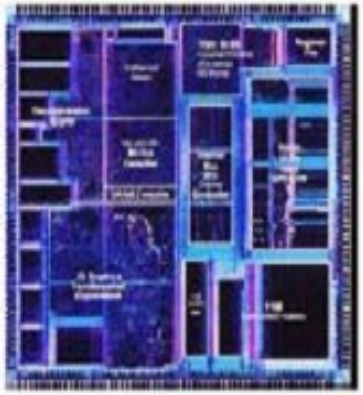
Assembly

System-Board

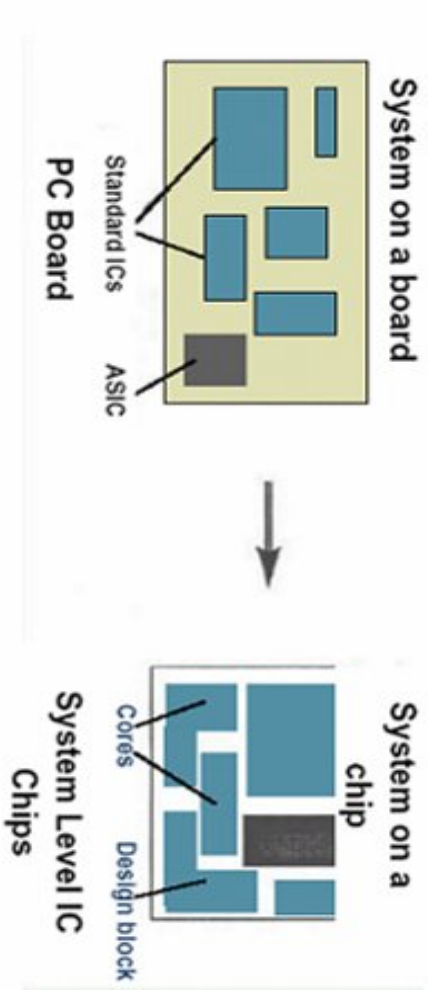
IP/System-Board

Tomorrow

Integration

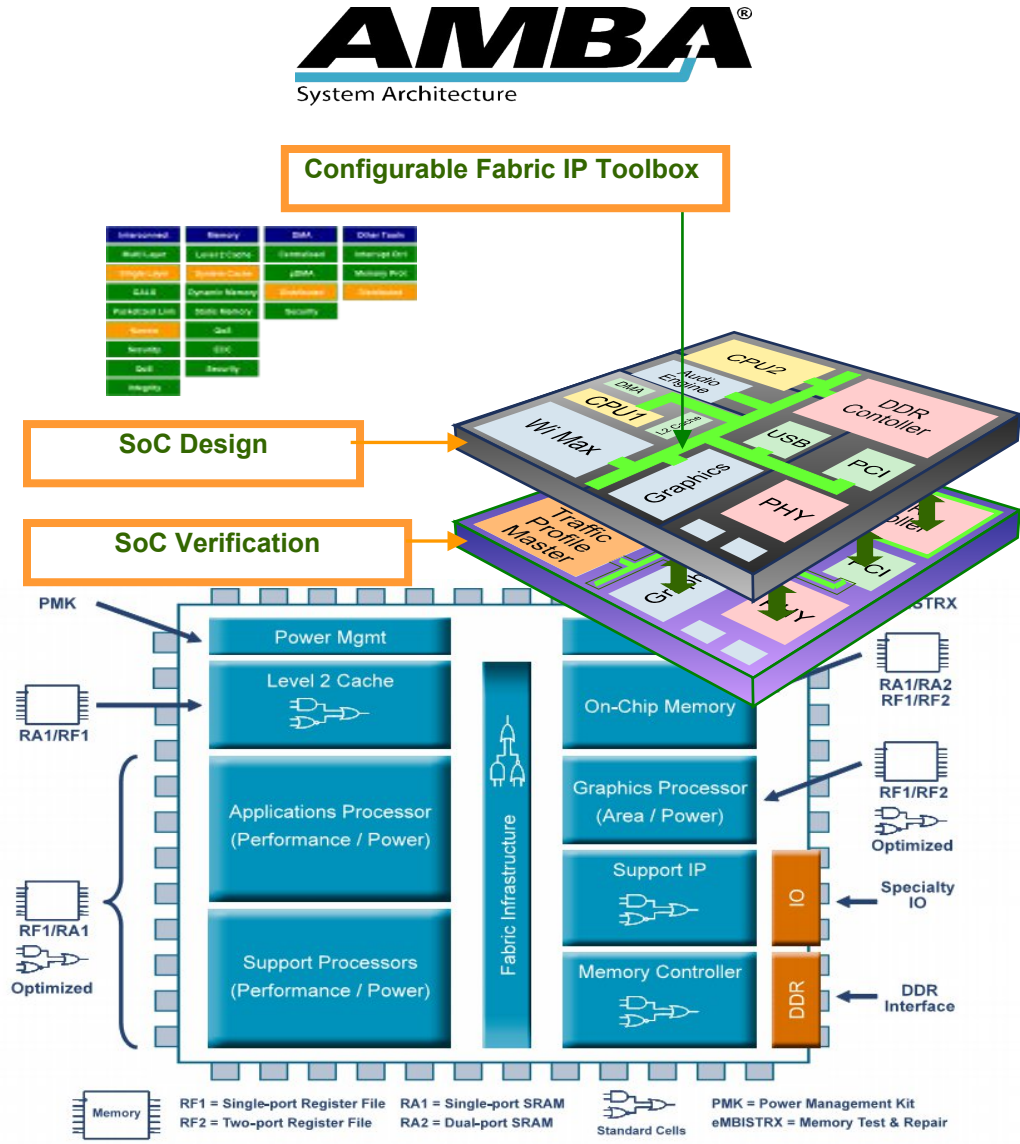


SOC

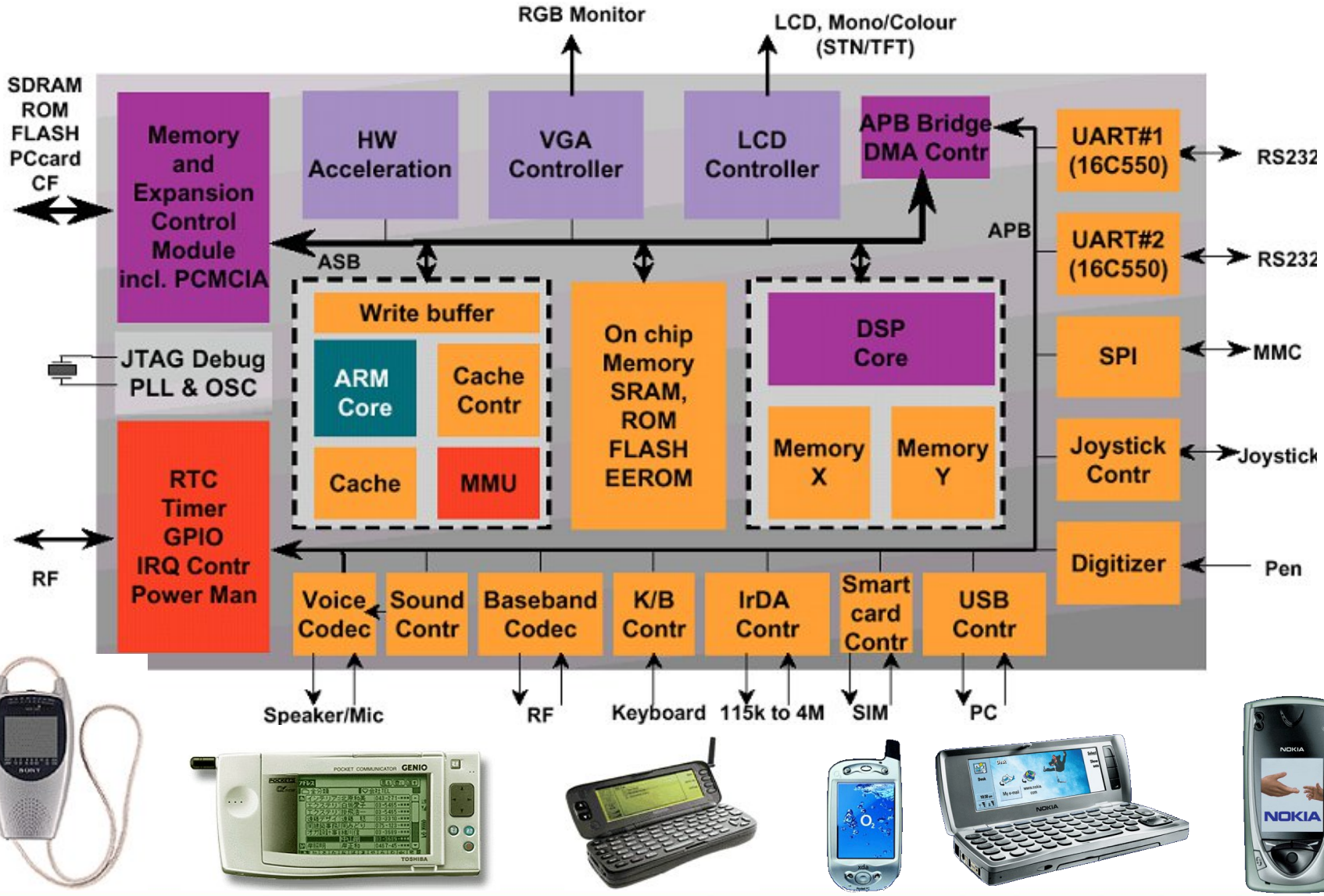


ARM's IP: Soc Development & Meet Specific Requirements

- **Besides common IPs, important to have your own IPs for specific applications and special purposes (security, flexibility, …).**
- **Reduced Time to Market and inceased efficiency.**
- **ARM's IPs provide solutions to:**
  - Embedded applications
  - Multimedia
  - Modems, routers
  - Processors
  - …



SoC Example

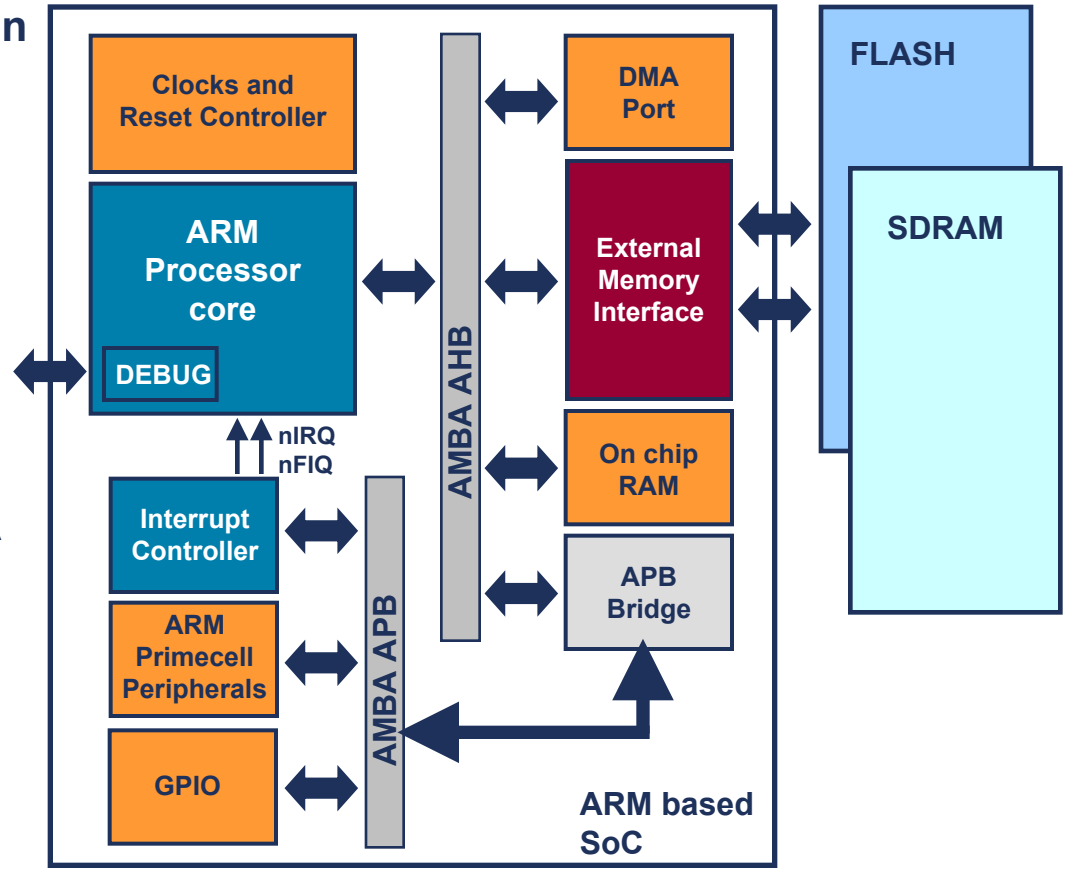

RGB Monitor
LCD, Mono/Colour (STN/TFT)
SDRAM
ROM
FLASH
PCcard
CF
Memory and Expansion Control Module incl. PCMCIA
HW Acceleration
VGA Controller
LCD Controller
APB Bridge DMA Contr
UART#1 (16C550)
RS232
UART#2 (16C550)
RS232
APB
ASB
Write buffer
ARM Core
Cache Contr
Cache
MMU
On chip Memory SRAM, ROM FLASH EEROM
DSP Core
Memory X
Memory Y
SPI
MMC
JTAG Debug
PLL & OSC
Joystick Contr
Joystick
RTC
Timer
GPIO
IRQ Contr
Power Man
RF
Digitizer
Pen
Voice Codec
Sound Contr
Baseband Codec
K/B Contr
IrDA Contr
Smart card Contr
USB Contr
Speaker/Mic
RF
Keyboard
115k to 4M
SIM
PC

Agenda

- Introduction to ARM Ltd
- **The ARM Architecture**
  - **Introduction**
  - Programmer's Model
  - Instruction Sets
- ARM Processor Cores
- Writing Software for ARM Processors

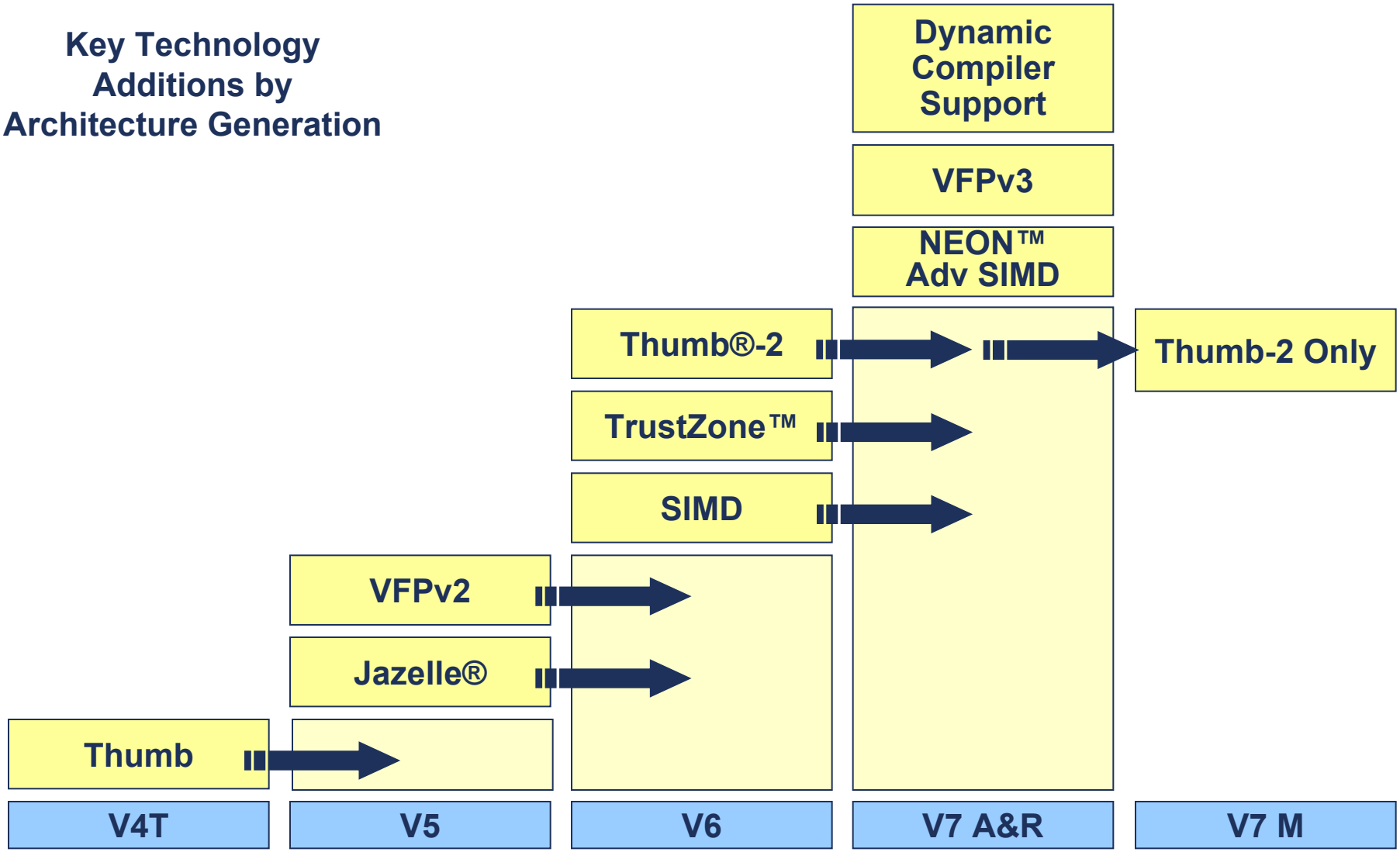
Example ARM based System

- **ARM core deeply embedded within a SoC**
  - External debug via JTAG port
- **Design has both external and internal memories**
  - Of varying width, speed and size
- **Includes an interrupt controller**
  - Core only support two interrupts
- **Includes Primecell peripherals**
  - Licensed from ARM
- **Elements connected using AMBA (Advanced Microcontroller Bus Architecture)**



ARM instruction set evolution

Key Technology Additions by Architecture Generation



ARM Architecture profiles

- **Application profile (ARMv7A)**
  - Memory management support (MMU)
  - Highest performance at low power
    - Influenced by multi-tasking OS system requirements
  - TrustZone and Jazelle-RCT for a safe, extensible system
  - e.g. Cortex-A8
- **Real-time profile (ARMv7R)**
  - Protected memory (MPU)
  - Low latency and predictability ‘real-time’ needs
  - Evolutionary path for traditional embedded business
  - e.g. Cortex-R4
- **Microcontroller profile (ARMv7M)**
  - Lowest gate count entry point
  - Deterministic behavior a key priority
  - Deeply embedded – strong synergies with ARMv7R
  - e.g. Cortex-M3

Which architecture is my processor?

Processor core	Architecture
ARM7TDMI family	**v4T**
ARM720T, ARM740T	
ARM9TDMI family	**v4T**
ARM920T,ARM922T,ARM940T	
ARM9E family	**v5TE, v5TEJ**
ARM946E-S, ARM966E-S, ARM926EJ-S	
ARM10E family	**v5TE, v5TEJ**
ARM1020E, ARM1022E, ARM1026EJ-S	
ARM11 family	**v6**
ARM1136J(F)-S	v6
ARM1156T2(F)-S	v6T2
ARM1176JZ(F)-S	v6Z
Cortex family	
ARM Cortex-A8	v7A
ARM Cortex-R4	v7R
ARM Cortex-M3	v7M

- **This presentation contains an Appendix**
  - Details ARM processor naming conventions and features

ARM Nomenclature

ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {S}

x => Family

y => memory management / protection unit

z => cache

T => Thumb 16-bit decoder

D => JTAG debugging

M = Fast multiplier

I => Enhanced ICE macrocell

E => Enhanced instructions (assumes TDMI)

J => Jazelle, hardware accelerated JAVA

F => Vector floating point unit

S => Synthesizable version

Agenda

- Introduction to ARM Ltd
- **The ARM Architecture**
  - Introduction
  - **Programmer's Model**
  - Instruction Sets
- ARM Processor Cores
- ARM System Design
- Writing Software for ARM Processors

Data Sizes and Instruction Sets

- **ARM is a RISC architecture**
  - Many instructions execute in a single cycle
- **ARM is a 32-bit load / store architecture**
- **When used in relation to the ARM:**
  - **Halfword** means 16 bits (two bytes)
  - **Word** means 32 bits (four bytes)
  - **Doubleword** means 64 bits (eight bytes)
- **Most ARMs implement two instruction sets**
  - 32-bit **ARM** Instruction Set
  - 16-bit **Thumb** Instruction Set
- **Latest ARM cores introduce a new instruction set Thumb-2**
  - Provides a mixture of 32-bit and 16-bit instructions
  - Maintains code density with increased flexibility
- **Jazelle cores can also execute Java bytecode**

Comparison between RISC & CISC

Sr. No	RISC	CISC
1	Simple instructions taking 1 cycle	Complex instructions taking multiple cycles
2	Only LOADS / STORES reference memory	Any instruction may reference memory
3	Highly pipelined	Not pipelined or less pipelined
4	Instructions executed by the hardware	Instructions interpreted by the microprogram
5	Fixed format instructions	Variable format instructions
6	Few instructions and modes	Many instructions and modes
7	Complexity is in the compiler	Complexity is in the microprogram (hardware complexity)
8	Multiple register sets	Single register set

RISC Design Philosophy

Four major **design rules**:

1. **Instructions:**
   - executing in single cycle
   - uniform and fixed length instructions => simple decoding
   - compiler / programmer synthesizes complicated operations by combining several simple instructions
2. **Pipelines:**
   - instructions are broken in simple and smaller steps
   - pipeline advance one step on each cycle ideally
3. **Registers:**
   - large general-purpose register set (containing data / address)
   - orthogonal registers (can be used for data and address)
4. **Load-store architecture:**
   - operands are stored in registers (reduced memory access)
   - load / store instructions transfer data between the register bank and external memory.

Data types

- ARM processor supports 6 data types
  - 8-bits signed and unsigned bytes
  - 16-bits signed and unsigned half-words, aligned on 2-byte boundaries
  - 32-bits signed and unsigned words, aligned on 4-byte boundaries
- ARM instructions are all 32-bit words, word-aligned

  Thumb instructions are half-words, aligned on 2-byte boundaries
- Internally all ARM operations are on 32-bit operands; the shorter data types are only supported by data transfer instructions. When a byte is loaded form memory, it is zero- or sign-extended to 32 bits
- ARM coprocessor supports floating-point values
- Each instruction can be viewed as performing a defined transformation of the states

  visible registers

  invisible registers

  system memory

  user memory

Additional ARM Technical Features

- **System-on-Chip (SoC) Compatibility:**
  - ARM IP licensing model supports SoC (placing several components like processor, memory, and peripherals on the same chip) applications.
- **Reduced Power Consumption:**
  - ARM requires less power due to its simple architecture and fewer registers as compared to most other RISC architectures.
  - Most ARM products have a *low power mode* => a power saving state initiated by the software when no operations performed.
  - Processor automatically returns to its normal state on interrupt
  - extends battery power => small size => suitable for mobile phones, PDAs etc.
- Use of low-cost memory devices
- Reduced die area => cost reduction w.r.t design and manufacturing
- Hardware debug technology implemented within the processor => helpful for software developers with greater visibility

Additional ARM Technical Features

- **Improved Code Density:**
  - Code density = a rough measure of how much work a processor can perform versus program size.
  - RISC architectures have low code density (sometimes more instructions needed than a CISC) => due to larger program image
  - ARMv4 uses *Thumb state* => permits execution of either 16-bit or 32-bit instructions. 16-bit instructions improve code density by 30% over 32-bit fixed length instructions. Compiled program is small, or smaller than that of a CISC machine.
  - *conditional instruction execution* => small *if-then* programming constructs implemented without a branch instruction => preserves instruction pipeline.
  - suitable for limited on-board memory devices like mobile phones, mass storage devices etc.
- Variable cycle execution for certain instructions e.g. load-store-multiple instructions vary in the number of execution cycles (register dependent)
- Inline barrel shifter: preprocesses one of the input registers before it is used by an instruction => improves core performance & code density

Processor Modes

- **The ARM has seven basic operating modes:**
  - Each mode has access to
    - Its own stack space and a different subset of registers
  - Some operations can only be carried out in a privileged mode

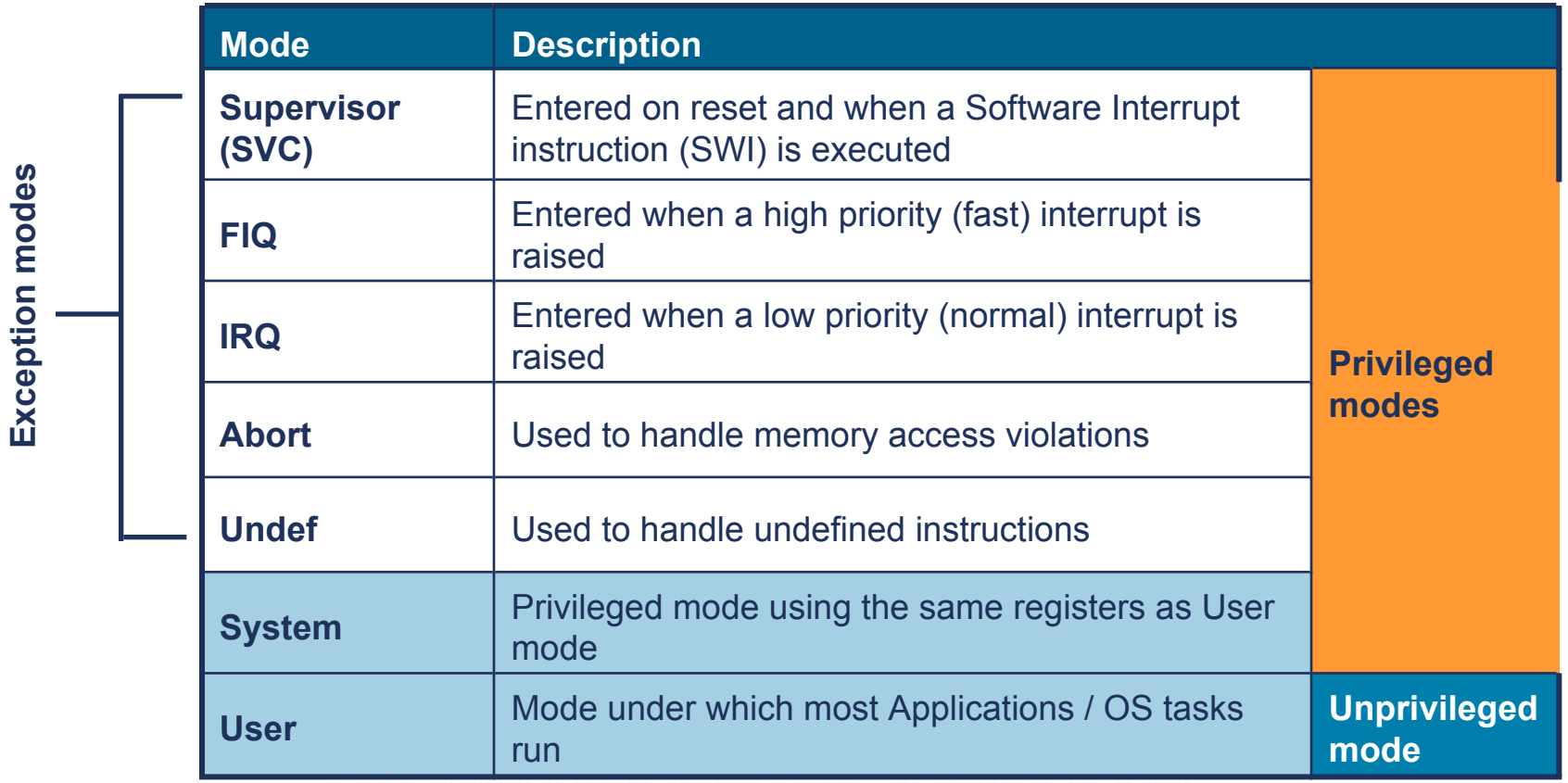
Modes	M[4:0]	Abv.	Description
User (User Program Mode)	10000	usr	Normal program execution mode *Access to system registers is not allowed (protected).* *Larger OSes use them for executing application level programs.*
FIQ (Fast Interrupt Mode)	10001	fiq	Used for fast interrupt processing. *High speed data transfer or channel process support due to the presence of registers r8_fiq - r12_fiq. Can be used without saving and restoring their context.*
IRQ (Normal Interrupt Mode)	10010	irq	Used for general purpose interrupt handling. *Does not have any scratch registers => so the s/w must save some of the register set prior to doing any IRQ processing.*
Supervisor (Supervisor Mode)	10011	svc	A protected mode for the operating system *Most embedded systems execute their programs in this mode (also a typical program execution mode)*

Processor Modes

Modes	M[4:0]	Abv.	Description
Abort	10111	abt	Implements virtual memory and/or memory protection *A program exception mode => for handling instruction fetch abort and data memory access abort conditions.* *The address of the abort routine is contained in the r14_abt register and the CPSR at the time of the abort can be found in the SPSR_abt register.*
Undefined	11011	und	Supports software emulation of hardware coprocessors *A program exception mode => for handling undefined instruction error conditions.* *The address of the undefined instruction is in the r14_und register and the CPSR at the time of the undefined instruction can be found in the SPSR_und register.*
System	11111	sys	Runs privileged operating system tasks (**ARMv4** and above) *A program execution mode => for typical embedded system Programs*

Processor Modes

- **The ARM has seven basic operating modes:**
  - Each mode has access to
    - Its own stack space and a different subset of registers
  - Some operations can only be carried out in a privileged mode



- Mode changes by software control or external interrupts

Processor Modes

- Mode changes by software control or external interrupts

Mode		Description
User	usr	Normal program execution
FIQ	fiq	Supports low latency and high-speed data transfer.
IRQ	irq	Used for general-purpose interrupt handling.
Supervisor	svc	A protected mode for the operating system.
Abort	abt	Implements virtual memory and/or memory protection.
Undefined	und	Supports software emulation of hardware coprocessors.
System	sys	Runs privileged operating system tasks.

Processor Modes

- **Mode changes can be made under software control or caused by exception processing.**
- **The processor modes are categorized into three categories:**
  - User mode.
  - Privilege mode.
    - FIQ, IRQ, Supervisor, Abort, Undefined.
  - System mode.

- Most programs operate in user mode. ARM has other privileges operating modes which are used to handle exceptions, supervisor calls (software interrupts), and system mode
- Current operating mode is defined by CPSR[4:0]

The Mode Bits

- Mode changes by software control or external interrupts

CPSR[4:0]	Mode	Use	Registers
10000	User	Normal user code	user
10001	FIQ	Processing fast interrupts	_fiq
10010	IRQ	Processing standard interrupts	_irq
10011	SVC	Processing software interrupts (SWIs)	_svc
10111	Abort	Processing memory faults	_abt
11011	Undef	Handling undefined instruction traps	_und
11111	System	Running privileged operating system tasks	user

Processor Modes: User Mode

- Most application programs run under the User mode.
- While the processor in in User mode, the user program being executed is unable to access some protected system resources.
- User mode programs cannot change mode, except for causing an exception to occur.
- The above stated restrictions of the User mode provide a mechanism for a suitably written operating system to control and manage the use of system resources.

Processor Modes: Privileged Mode

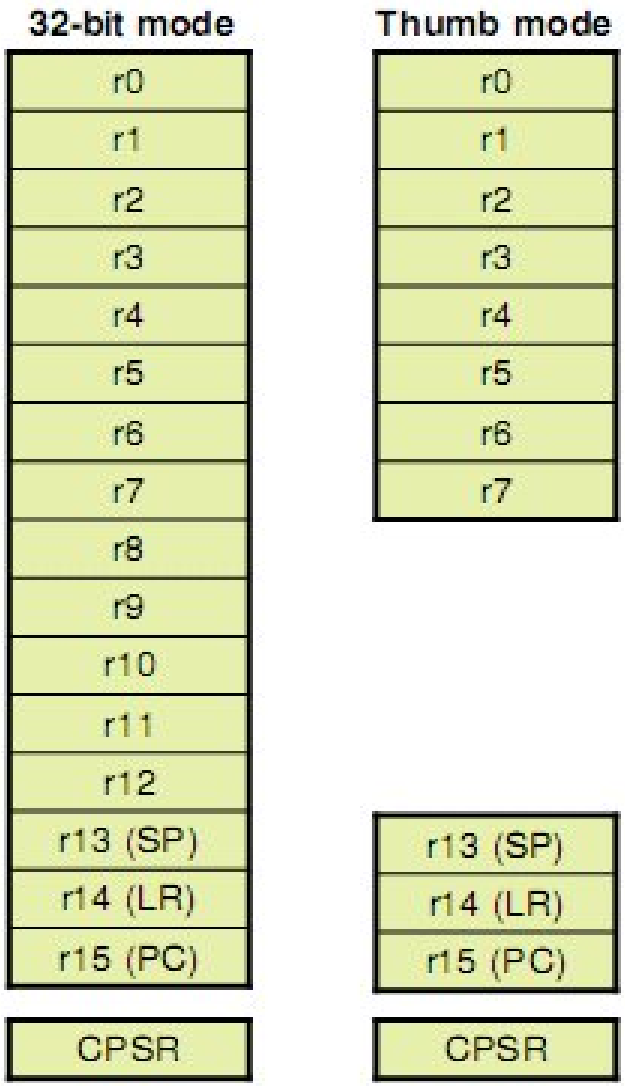
- **Whenever an exception occurs, the processor enters one of the privileged modes:**
  - FIQ, IRQ, Supervisor, Abort, Undefined.
- **They are entered when the specific exceptions occur.**
- **Each has their own set of banked registers to avoid corrupting the previous mode's registers when an exception occurs.**
- **They have full access to the system resources.**
- **They can change modes freely.**

Processor Modes: System Mode

- **System mode is not entered via an exception.**
- **It has the same registers as in the User mode.**
- **It has the privilege mode, so it can access system resources to the same extent as the Privileged modes.**
- **It is intended to be used by the operating system for system resource access, without having to generate an exception (to enter one of the privileged modes).**
  - In this way, privilege mode access is entered without having to use the additional registers associated with the privileged modes.
  - Thus, this avoids the storing the additional registers to the stack.

- Having some protective privileges
- System-level function (transaction with the outside world) can be accessed through specified supervisor calls
- Usually implemented by software interrupt (SWI)

Registers

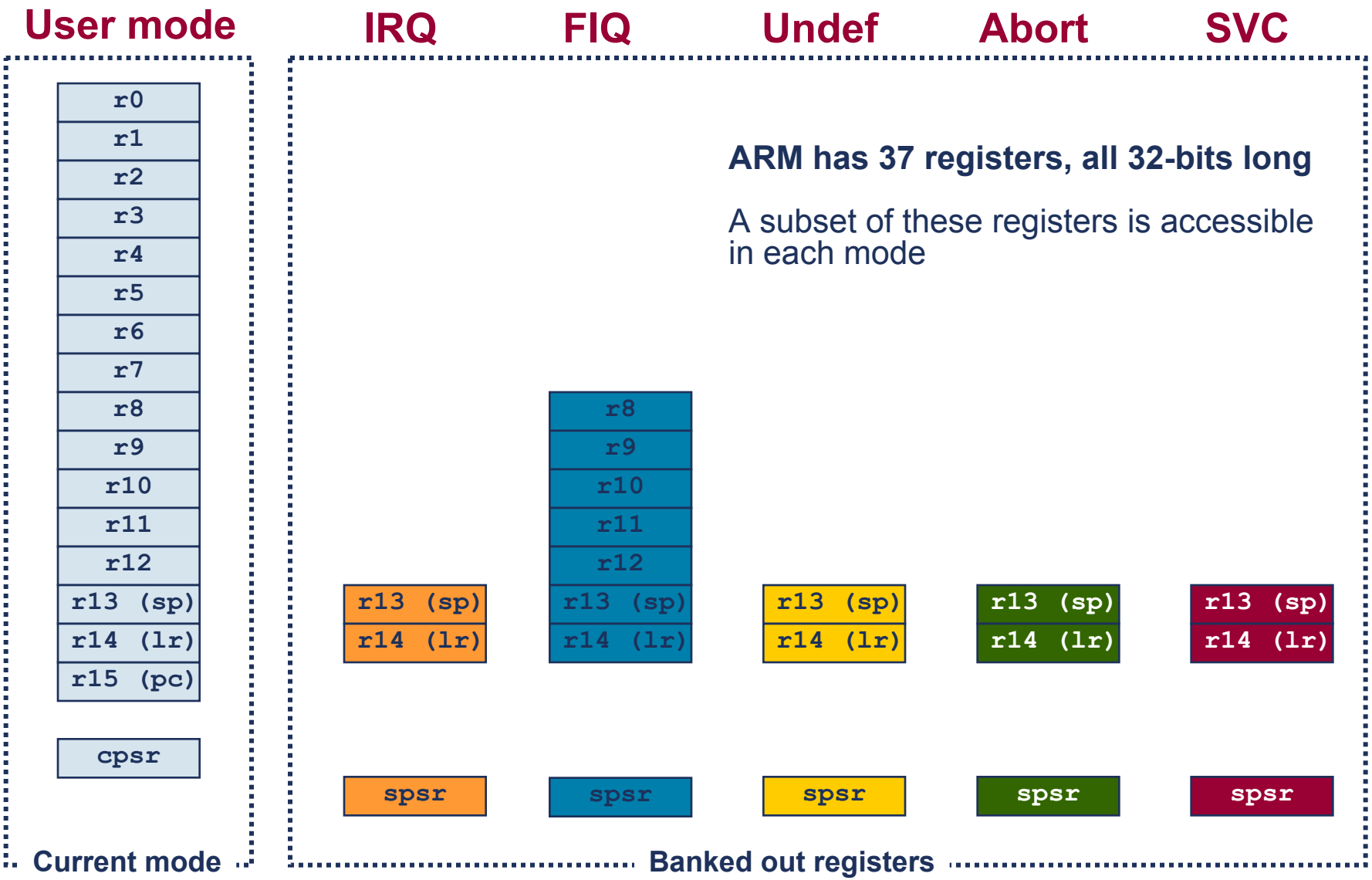


- Total of 37 32-bit registers (in several banks):
  - one PC
  - one Current Program Status Register (CPSR)
  - five Saved Program Status Registers (SPSR)
  - 30 general-purpose registers
- Processor modes determine Register access
- Each processor mode can access
  - a particular set of general-purpose registers (r0 through r12)
  - a particular Stack Pointer (r13)
  - a particular Link Register (r14)
  - the PC (r15), and
  - either the CPSR or the SPSR for that mode.
- Registers r0 through r7 => Thumb state application programming.
- Register r14 is the Link Register (LR):
  - saves the return address on function calls
  - stores the point of interrupt or exception.
- Register r13 is the Stack Pointer (SP)
  - points to the top of the stack
  - grows toward lower memory addresses.

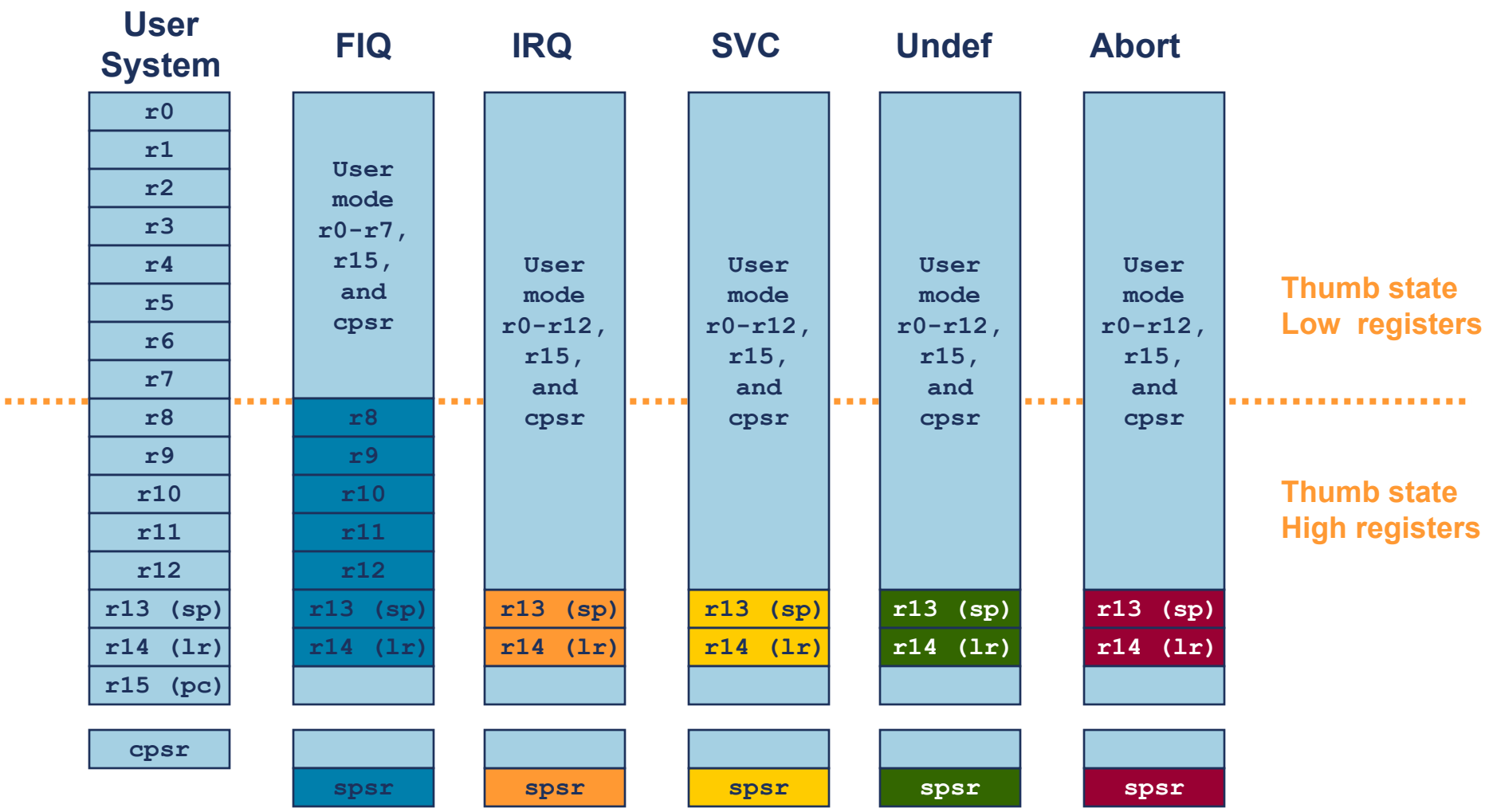
General Purpose Registers

- **The unbanked registers**
  - `r0 – r15`
  - user and system mode refer to the same physical registers
- **The banked registers**
  - `r8_fiq – r12_fiq`, `r13_`<mode>, and `r14_`<mode>
  - The set of physical registers depend on the processor mode
  - `r13` is normally used as the stack pointer (SP)
  - `r14` is also known as the link register (LR), which is used to store the return address from a subroutine
- **Register 15, PC**
  - `r15` is the program counter

The ARM Register Set



Register Organization Summary



Note: System mode uses the User mode register set

Program Counter (r15)

- **When the processor is executing in ARM state:**
  - All instructions are 32 bits wide
  - All instructions must be word aligned
  - Therefore the **pc** value is stored in bits [31:2] with bits [1:0] undefined (as instruction cannot be halfword or byte aligned)

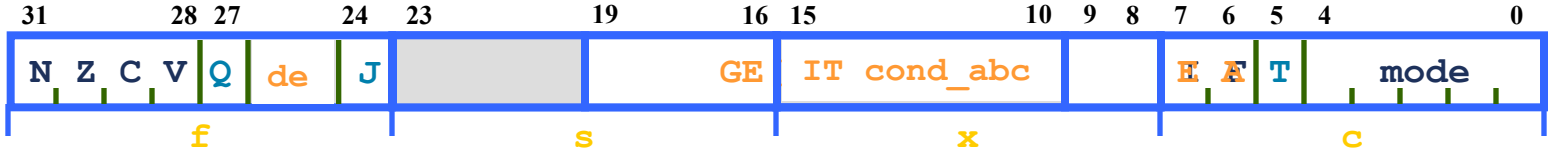
- **When the processor is executing in Thumb state:**
  - All instructions are 16 bits wide
  - All instructions must be halfword aligned
  - Therefore the **pc** value is stored in bits [31:1] with bit [0] undefined (as instructions cannot be byte aligned)

CPSR Register Values

31	30	29	28	27		24		7	6	5	4	3	2	1	0
N	Z	C	V	Q		J		I	F	T	M	M	M	M	M

CPSR bit	Description
	Condition Code (Status) Flags
N	N = 1 => when result is negative after an instruction
Z	Z = 1 => when result is zero after an instruction
C	C = 1=> when a carry or borrow occurs after an instruction
V	V = 1=> when an overflow occurs after an instruction
Q	Q = 1=> when an overflow and / or saturation occurs in an enhanced DSP instruction
	Interrupt Disable Bits
I	I = 1 => IRQ interrupts disabled
F	F = 1 => FIQ interrupts disabled
	T Bit (Architecture v4T only)
T	If set, the Thumb 16-bit instruction set is in use. This bit cannot be set by writing directly into the CPSR. See the BX instruction in the ARM Architecture Reference Manual.
	J Bit (Jazelle enabled cores only)
J	If set, the core is in Jazelle state. J bit is generally not usable (available only on some cores). Extra software license needed from both ARM and Sun Microsystems for Jazelle.
	Mode bits M[4:0]
MMMMM	Processor mode bits

Program Status Registers



- **Condition code flags**
  - N = **N**egative result from ALU
  - Z = **Z**ero result from ALU
  - C = ALU operation **C**arried out
  - V = ALU operation o**V**erflowed
- **Sticky Overflow flag - Q flag**
  - Architecture 5TE and later only
  - Indicates if saturation has occurred
- **J bit**
  - Architecture 5TEJ and later only
  - J = 1: Processor in Jazelle state
- **Interrupt Disable bits.**
  - I = 1: Disables the IRQ
  - F = 1: Disables the FIQ

- **T Bit**
  - T = 0: Processor in ARM state
  - T = 1: Processor in Thumb state
  - Introduced in Architecture 4T
- **Mode bits**
  - Specify the processor mode

- **New bits in V6**
  - **GE[3:0]** used by some SIMD instructions
  - **E** bit controls load/store endianness
  - **A** bit disables imprecise data aborts
  - **IT [abcde]** IF THEN conditional execution of Thumb2 instruction groups

SPSRs

- Each privileged mode (except system mode) has associated with it a Save Program Status Register, or SPSR
- This SPSR is used to save the state of CPSR (Current Program Status Register) when the privileged mode is entered in order that the user state can be fully restored when the user process is resumed
- Often the SPSR may be untouched from the time the privileged mode is entered to the time it is used to restore the CPSR, but if the privileged supervisor calls to itself) then the SPSR must be copied into a general register and saved

I/O System & Interrupts

- ARM handles input/output peripherals as memory-mapped with interrupt support
- Internal registers in I/O devices as addressable locations within ARM's memory map

  read and written using load-store instructions
- Interrupt by normal interrupt (IRQ)

  or fast interrupt (FIQ) higher priority

  input signals are level-sensitive and maskable
- May include Direct Memory Access (DMA) hardware

Synchronous & Asynchronous Interrupts

- **Synchronous**
  - Produced by the processor while executing instructions.
  - Issues only after finishing execution of an instruction.
  - Often called exceptions.
  - Example: SWI, page faults, system calls, divide by zero

- **Asynchronous**
  - Generated by other hardware devices.
  - Occur at arbitrary times, including while CPU is busy executing an instruction.
  - Ex: I/O, timer interrupts

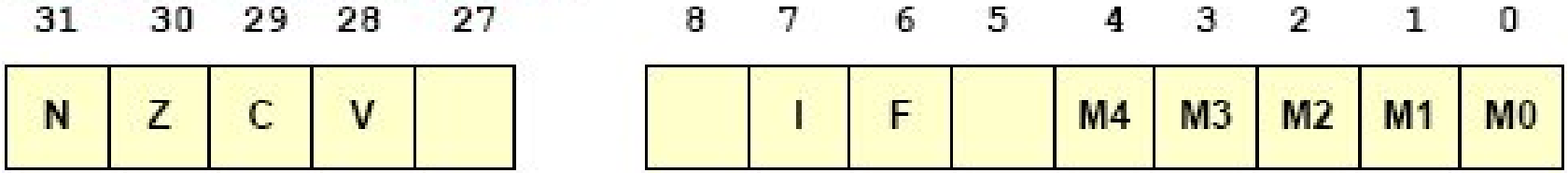
ARM Interrupts

- **Vector table**
  - Reserved area of 32 bytes at the end of the memory map
  - One word of space for each exception type
  - Contains a Branch or Load PC instruction for the exception handler

- **Exception modes and registers**
  - Handling exceptions changes program from user to non-user mode
  - Each exception handler has access to its own set of registers
    - Its own r13 = stack pointer
    - Its own r14 = link register
    - Its own SPSR (Saved Program Status Register)
  - Exception handlers must save (restore) other register on entry (exit)

Enabling IRQ and FIQ

- **Program Status Register**



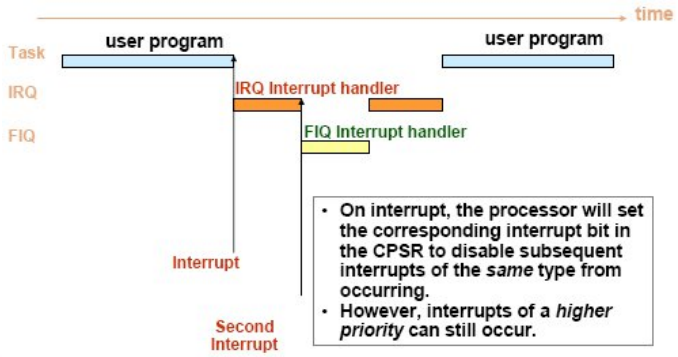
- **To disable interrupts, set corresponding "F" or "I" bit to 1**
- **On interrupt, processor does the following**
  - Switches register banks
  - Copies CPSR to SPSR_mode (saves mode, interrupt flags, etc.)
  - Changes the CPSR mode bits (M[4:0])
  - Disables interrupts
  - Copies PC to R14_mode (to provide return address)
  - Sets the PC to the vector address of the exception handler
- **Interrupt handlers must contain code to clear the source of the interrupt**

Interrupt Handling

- **On an IRQ interrupt, the ARM processor will ...**
  - If the “I”bit in the CPSR is clear, the current instruction is completed and then the processor will
  - Save the address of the next instruction plus 4 in r14_irq
  - Save the CPSR in the SPSR_irq
  - Force the CPSR mode bits M[4:0] to 10010 (binary)
- **This switches the CPU to IRQ mode and then set s the “I”flag to disable further IRQ interrupts**
- **On an FIQ interrupt, the processor will ...**
  - If the “F”bit in the CPSR is clear and the current instruction is completed, the ARM will
  - Save the address of the next instruction plus 4 in r14_fiq
  - Force the CPSR mode bits M[4:0] to 10001 (binary)
    - This switches the CPU to FIQ mode and then sets the “I”and “F”flags to disable further IRQ or FIQ interrup

IRQ vs. FIQ

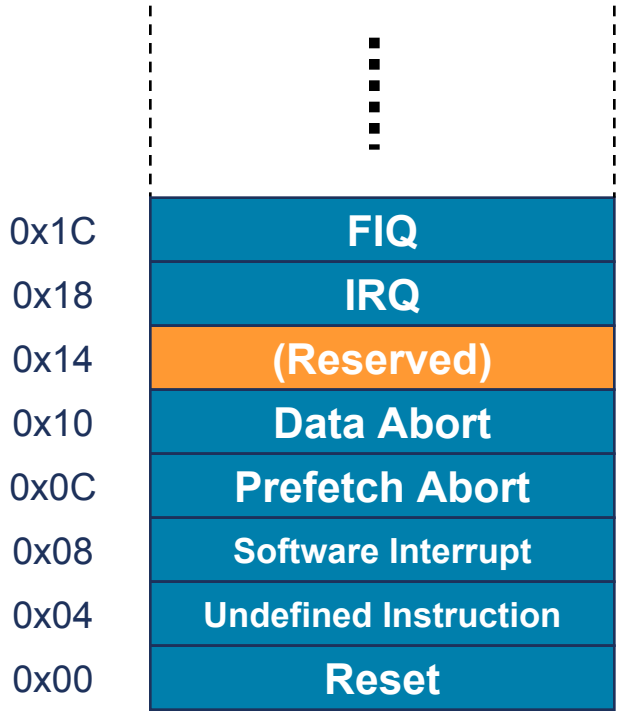
- **FIQs have higher priority than IRQs**
  - When multiple interrupts occur, FIQs get serviced before IRQs
  - Servicing an FIQ causes IRQs to be disabled until the FIQ handler re-enables them
    - CPSR restored from the SPSR at the end of the FIQ handler
- **How are FIQs made faster?**
  - They have five extra registers at their disposal, allowing them to store status between calls to the handler
  - FIQ vector is the last entry in the vector table
    - The FIQ handler can be placed directly at the vector location and run sequentially a fter the location
    - Cache-based systems: Vector table + FIQ handler all locked down into one block



Exception Handling

- **When an exception occurs, the core:**
  - Copies CPSR into SPSR_<mode>
  - Sets appropriate CPSR bits
    - Change to ARM state
    - Change to exception mode
    - Disable interrupts (if appropriate)
  - Stores the return address in LR_<mode>
  - Sets PC to vector address
- **To return, exception handler needs to:**
  - Restore CPSR from SPSR_<mode>
  - Restore PC from LR_<mode>

  This can only be done in ARM state



Vector Table

Vector table can be at **`0xFFFF0000`** on ARM720T and later

Exception Entry

- When an exception arises, ARM completes the current instruction as best it can (except that reset exception terminates the current instruction immediately) and then departs from the current instruction sequence to handle the exception which starts from a specific location (exception vector)
- Processor performs the following sequence
  - change to the operating mode corresponding to the particular exception
  - save the address of the instruction following the exception entry instruction in r14 of the new mode
  - save the old value of CPSR in the SPSR of the new mode
  - disable IRQs by setting bit of the CPSR, and if the exception is a fast interrupt, disable further faster interrupt by setting bit of the CPSR
  - force the PC to begin executing at the relevant vector address

Exception	Mode	Vector address
Reset	SVC	0x00000000
Undefined instruction	UND	0x00000004
Software interrupt (SWI)	SVC	0x00000008
Prefetch abort (instruction fetch memory fault)	Abort	0x0000000C
Data abort (data access memory fault)	Abort	0x00000010
IRQ (normal interrupt)	IRQ	0x00000018
FIQ (fast interrupt)	FIQ	0x0000001C

- Normally the vector address contains a branch to the relevant routine, though the FIQ code can start immediately
- Two banked registers in each of the privilege modes are used to hold the return address and stack pointer

Exception Return

- Once the exception has been handled, the user task is normally resumed
- The sequence is
  - any modified user registers must be restored from the handler'stack
  - CPSR must be restored from the appropriate SPSR
  - PC must be changed back to the relevant instruction address
- The last two steps happen atomically as part of a single instruction
- When the return address has been kept in the banked r14
  - to return from a SWI or undefined instruction trap

    ```
    MOVS PC,r14
    ```

  - to return from an IRQ, FIQ or prefetch abort

    ```
    SUBS PC,r14,#4
    ```

  - To return from a data abort to retry the data access

    ```
    SUBS PC,r14,#8
    ```

  - 'S' signifies when the destination register is the PC
- When the return address has been saved onto a stack

  ```
  LDMFD r13!,{r0-r3,PC}^      ;restore and return
  ```

  - '^' indicates that this is a special form of the instruction the CPSR is restored at the same time that the PC is loaded from memory, which will always be the last item transferred from memory since the registers are loaded in increasing order

ARM Exceptions / Interrupts

Generally the following steps are followed for handling exceptions:

1. The current state is saved by copying the PC into rl4_exc and the CPSR into SPSR_exc (where exc stands for the exception type).
2. The processor operating mode is changed to the appropriate exception mode.
3. The PC is forced to a value between 00_{16} and $1C_{16}$, the particular value depending on the type of exception

Exception / Interrupt	Abbreviation	Address	High Address
Reset	RESET	0x00000000	0xffff0000
Undefined Instruction	UNDEF	0x00000004	0xffff0004
Software Interrupt	SWI	0x00000008	0xffff0008
Prefetch Abort	PABT	0x0000000c	0xffff000c
Data Abort	DABT	0x00000010	0xffff0010
Reserved	----	0x00000014	0xffff0014
Interrupt Request	IRQ	0x00000018	0xffff0018
Fast Interrupt Request	FIQ	0x0000001c	0xffff001c

Exception Priority

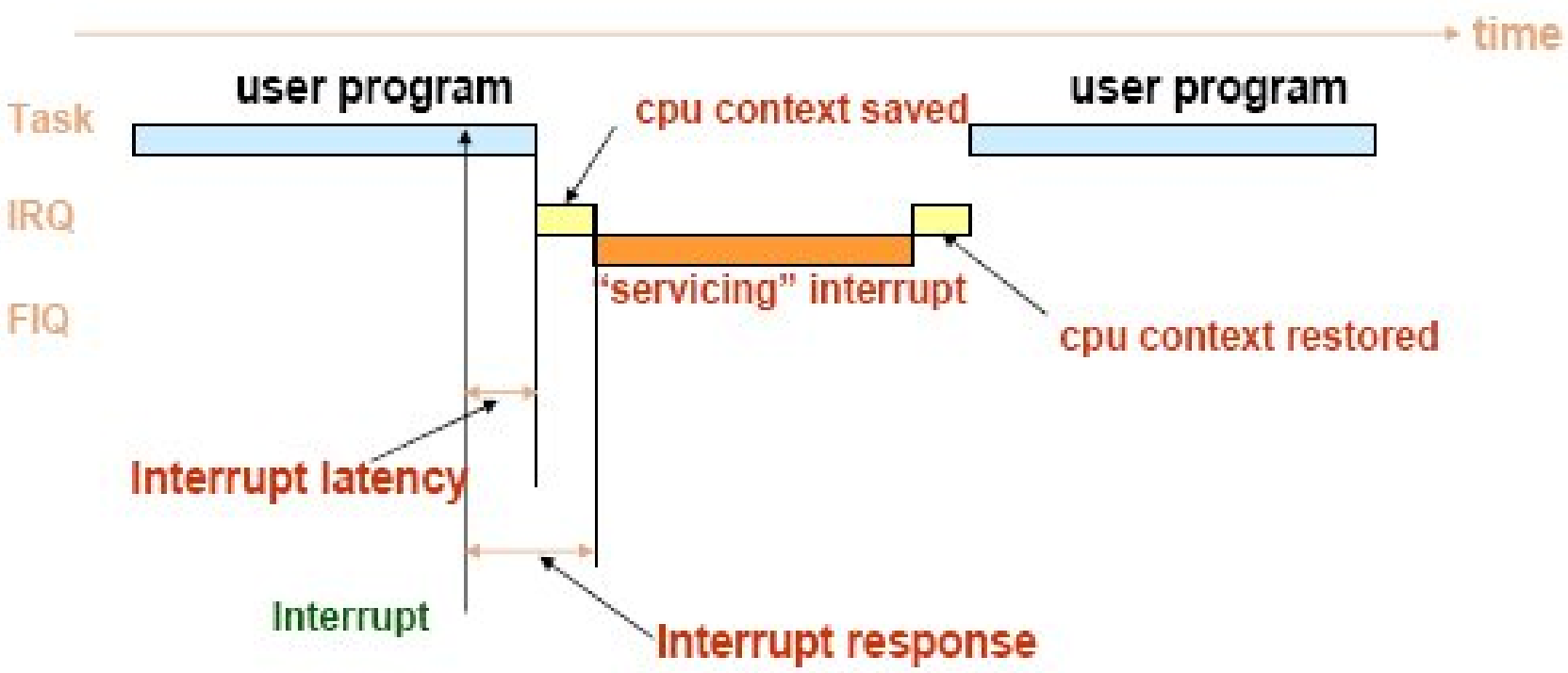
- Priority order
  - reset (highest priority)
  - data abort
  - FIQ
  - IRQ
  - prefetch abort
  - SWI, undefined instruction

Non/Auto-Vectored and Vectored Interrupts

- **Auto-vectored**
  - Processor-determined address of interrupt handler based on type of interrupt
- **Vectored**
  - Device supplies processor with address of interrupt handler
- **Why the different methods?**
  - If multiple devices uses the same interrupt type (IRQ vs. FIQ), in an Auto-vectored s ystem the processor must poll each device to determine which device interrupted the processor
    - This can be time-consuming if there is a lot of devices
  - In a vectored system, the processor would just take the address from the device (whi ch dumps the interrupt vector onto a special bus).

Interrupt Timing (Latency & Context Switch)

- **Before an interrupt handler can do anything, it must save away the current program's registers (if it touch es those registers)**
  - That's why the FIQ has lots of extra registers to minimize CPU co ntext saving overhead



How To Implement Long Interrupts

- **Interrupt handling sometimes needs to perform lengthy tasks.**
  This problem is resolved by splitting the interrupt handler into two halves:
- **Top half responds to the interrupt**
  - The one registered to request_irq
  - Saves data to device-specific buffer and schedules the bottom half
  - Current interrupt disabled, possibly all disabled.
  - Runs in interrupt context, not process context. Can't sleep.
  - Acknowledges receipt of interrupt.
  - Schedules bottom half to run later.
- **Bottom half is scheduled by the top half to execute later**
  - With all interrupts enabled
  - Wakes up processes, starts I/O operations, etc.
  - Runs in process context with interrupts enabled.
  - Performs most work required. Can sleep.
  - Ex: copies network data to memory buffers.

How To Implement Long Interrupts

- **Three mechanisms may be used to implement bottom halves**
  - SoftIRQs
    - Have strong locking requirements
    - Only used of performance sensitive subsystems –networking, SCSI, etc.
    - Reentrant
  - Tasklets
    - Built on top of SoftIRQs
    - Should not sleep
    - Cannot run in parallel with itself
    - Can run in parallel with other tasklets on SMP systems
    - Guaranteed to run on the same CPU that first scheduled them
  - Workqueues
    - Can sleep
    - Cannot copy data to and from user space

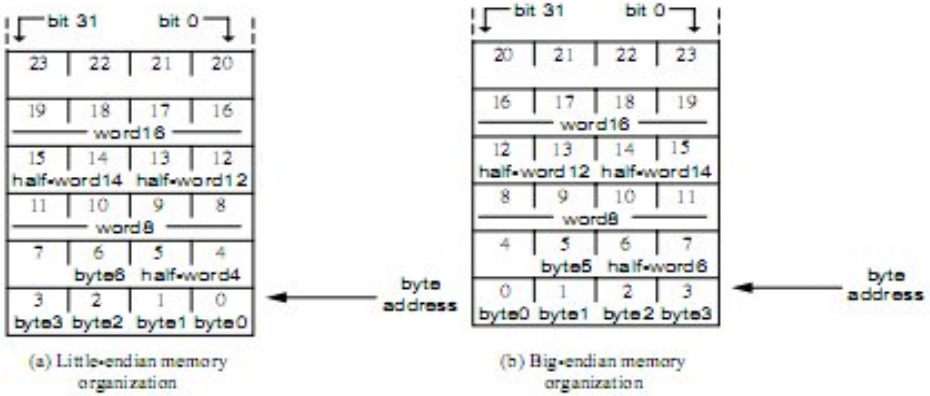
Dos and Don’ts of Interrupt Handlers

- It's a programming offense if your interrupt context code goes to sleep. Interrupt handlers cannot relinquish the processor by calling sleepy functions such as schedule_timeout().
- For protecting critical sections inside interrupt handlers, you can't use mutexes because they may go to sleep. Use spinlocks instead, and use them only if you must.
- Interrupt handlers are supposed to get out of the way quickly but are expected to get the job done. To circumvent this Catch-22, interrupt handlers split their work into two halves: top (slim) and bottom (fat).
- You do NOT need to design interrupt handlers to be reentrant. When an interrupt handler is running, the corresponding IRQ is disabled until the handler returns.
- Interrupt handlers can be interrupted by handlers associated with IRQs that have higher priority. You can prevent this nested interruption by specifically requesting the kernel to treat your interrupt handler as a fast handler.

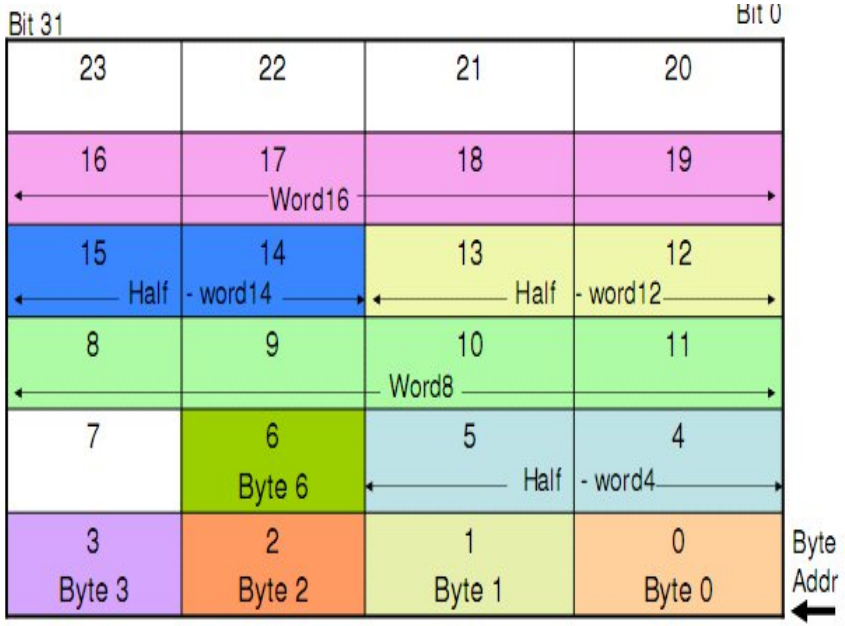
ARM Memory Model

- Linear array of bytes numbered from zero up to $(2^{32} - 1)$.
- Data items:
  - 8-bit bytes
  - 16-bit half-words or 32-bit words.
- Words are always aligned on 4-byte boundaries (that is, the two least significant address bits are zero)
- Half-words are aligned on even byte boundaries.
- Each byte location has a unique number and a byte may occupy any of these locations
- A word-sized data item occupies a group of four byte locations starting at a byte address which is a multiple of four.
- Half-words occupy two byte locations starting at an even byte address

ARM Memory Model



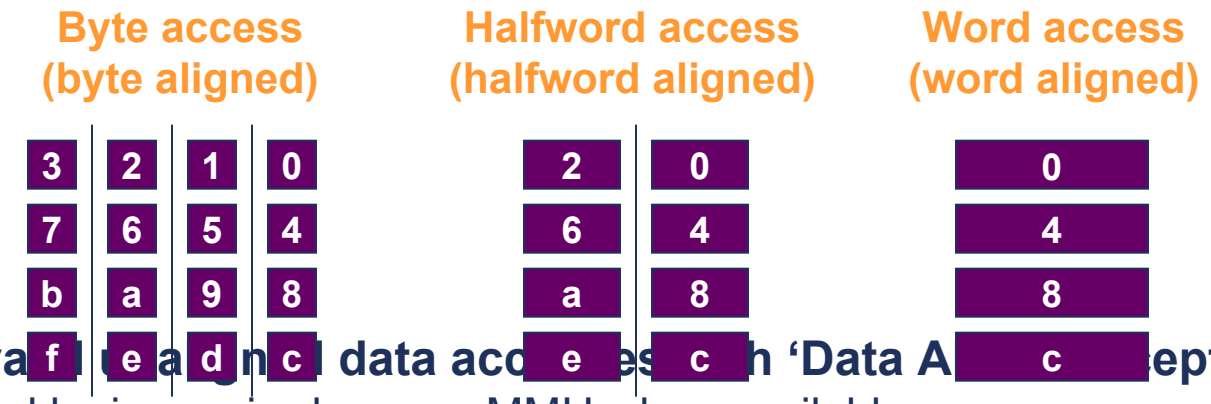
- Word, half-word alignment (xxxxoo or xxxxxo)
- ARM can be set up to access data in either little-endian or big-endian format



A Standard Little Endian memory Organization

Data Alignment

- **Memory accesses must always be appropriately aligned for size of access**
  - Unaligned addresses will produce unexpected/undefined results



- **Detect invalid unaligned data accesses with 'Data Abort' exception**
  - External logic required, or use MMU when available
  - Beware that instruction fetches may appear unaligned
- **Unaligned data can be accessed using aligned access combined with shift/mask operations**